Spread Prediction for Portfolio Optimisation

Evaluating whether spread prediction between cointegrated asset pairs produces more reliable return estimates than historical means for Markowitz optimisation

Declan Loo

BSc Computer Science

Supervisor's name: Dr. Tomaso Aste

Submission date: 24th April 2026

Disclaimer: This report is submitted as part requirement for the BSc Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Summarise your report concisely.

Contents

Chapter 1

Introduction

This project uses only publicly available equity price data and does not involve human subjects, personal data, or data collection. No ethical approval was required for the project.

1.1 Motivation

Markowitz mean-variance optimisation **markowitz1959** provides a rigorous framework for constructing efficient portfolios, yet its practical performance is highly sensitive to the accuracy of expected return estimates. Historical mean returns are extremely noisy estimators of future returns, with standard errors that often exceed the estimates themselves **ledoit2004**, causing optimisers to produce unstable, concentrated allocations that perform poorly out-of-sample. Cointegration-based pairs trading strategies exploit long-run equilibrium relationships between asset prices to generate mean-reverting spread signals **gatev2006**. While extensive literature exists on pairs trading as a standalone strategy, few studies integrate spread-based predictions directly into the MPT framework as a systematic approach to return estimation. This project addresses this gap by investigating whether the mean-reverting dynamics of cointegrated spreads, modelled via the Ornstein-Uhlenbeck process, can provide more reliable return signals for Markowitz optimisation than traditional historical means, thereby improving out-of-sample portfolio performance.

1.2 Research Question

Can spread prediction between cointegrated asset pairs provide more reliable return estimates for Modern Portfolio Theory optimisation than traditional historical mean approaches?

1.3 Research Hypothesis

Spread-based return estimation derived from cointegrated asset pairs produces superior portfolio allocations compared to historical mean-variance optimisation, measured by risk-adjusted returns and portfolio stability, in terms of Sharpe ratio, volatility reduction and maximum drawdown.

1.4 Aims and Objectives

The project has the following objectives:

- Develop a spread-based return estimation approach using cointegration analysis of asset pairs that provides more stable signals than historical mean returns.
- Build an integrated portfolio optimisation system using Markowitz framework and validate through comprehensive back testing against traditional mean-variance approaches using Sharpe ratio, volatility reduction, and drawdown metrics.
- Create an interactive interface to demonstrate real-world applicability for portfolio managers.

1.5 Report Overview

This report is structured as follows. Chapter **??** provides the theoretical foundations underlying the project, covering Modern Portfolio Theory, the return estimation problem, cointegration theory, and the Ornstein-Uhlenbeck process. Chapter **??** presents the methodology, detailing the cointegration screening procedure, spread modelling, OU-implied return estimation, and the minimum-variance portfolio optimisation framework. Chapter **??** describes the implementation of the four-notebook pipeline, portfolio optimisation module, backtesting engine, and dashboard interface. Chapter **??** presents the empirical results across cointegration screening, spread characterisation, out-of-sample validation, return estimation, and portfolio backtesting. Chapter **??** interprets these results, addressing the central research question and examining the role of spread universe structure in driving performance. Chapter **??** summarises the achievements of the project, provides a critical evaluation, and outlines directions for future work.

Chapter 2

Background and Literature Review

This chapter reviews foundational theories underpinning the pairs trading strategy. Key concepts include Markowitz portfolio optimisation **markowitz1959**, the return estimation problem, statistical arbitrage via pairs trading **vidyamurthy2004**, cointegration (Engle-Granger) **englegranger1987**, and the Ornstein–Uhlenbeck process for spread dynamics **elliott2005**. These establish the theoretical framework for the methodology in Chapters **??** – **??**.

2.1 Mean-Variance Portfolio Optimisation

Markowitz introduced Modern Portfolio Theory (MPT) in his seminal work *Portfolio Selection: Efficient Diversification of Investments* **markowitz1959**, establishing a quantitative framework for constructing optimal portfolios. MPT remains one of the most influential contributions to portfolio theory and optimisation, providing the theoretical foundation upon which much of modern finance is built. The theory characterises portfolio performance using two statistical measures: the mean as a proxy for expected return, and the standard deviation as a measure of risk. Within this framework, Markowitz defines a portfolio as **inefficient** if it is possible to achieve a higher expected return without increasing risk, or to reduce risk without sacrificing expected return. This gives rise to the *Efficient Frontier*, the set of portfolios that offer the maximum expected return for a given level of risk, where any portfolio lying below the curve is considered suboptimal. The selection of an optimal portfolio along the frontier is determined by the investor's individual risk-return preference, reflecting the principle that higher expected returns are generally associated with greater risk. Central to MPT is also the principle of diversification: by combining assets whose returns are not perfectly correlated, investors can reduce overall portfolio risk without proportionally reducing expected return, since assets that do not move in perfect unison partially offset each other's fluctuations. This shifted the focus of portfolio construction from selecting individually attractive assets to engineering a portfolio whose constituent asset interactions collectively minimise risk — a conceptual shift that underpins virtually all modern asset allocation practice.

However, MPT is not without limitations. Markowitz himself identified a fundamental shortcoming: the use of variance as the measure of risk was a pragmatic choice rather than a theoretically superior one. In his 1959 book, he explicitly argued that analyses based on semi-variance tend to produce *"better portfolios"* than those based on variance **markowitz1959**, since semi-variance

captures only downside deviations — the losses that investors genuinely seek to avoid — whereas variance penalises upside and downside movements equally. Nevertheless, variance was retained on the grounds that it held an advantage over semi-variance *"with respect to cost, convenience, and familiarity"* **markowitz1959**, particularly given the computational constraints of the era. A further, and arguably more practical, limitation of MPT is the instability of return estimation **fabozzi2002**. Fabozzi, Gupta, and Markowitz demonstrate empirically that historical return estimates are highly sensitive to the time period selected: for instance, the MSCI Emerging Markets index returned 16.3% annually over 1991–1995, yet only 0.1% over 1996–2000 **fabozzi2002**. The authors conclude that *"there is no right answer because we are dealing with the world of uncertainty"* **fabozzi2002**. Notwithstanding these limitations, they affirm that MPT *"will occupy a permanent place in the theory and practice of finance"* **fabozzi2002**.

2.2 The Return Estimation Problem

Historical mean returns are extremely noisy estimators of future returns, with standard errors that often exceed the estimates themselves **ledoit2004**. As Merton (1980) demonstrates, even assuming a constant expected return, it would take a very long history of returns to obtain an accurate estimate due to the high volatility-to-sample-size ratio **merton1980**.

Brealey (1969) provides a foundational decomposition of stock price movements into four influences: (1) market-wide, (2) industry-specific, (3) subgroup, and (4) firm-specific factors. This suggests that stocks within the same industry share both market and industry systematic influences, making sector-paired stocks natural candidates for cointegration analysis **brealey1969**.

2.3 Pairs Trading and Statistical Arbitrage

2.4 Cointegration Theory

Engle and Granger introduced the concept of Cointegration where pairs that follow I(1) are deemed stationary if they pass the hypothesis tests deemed as the Engle-Granger test **englegranger1987**. A study by

2.5 The Ornstein-Uhlenbeck Process

Elliott in his Pairs Trading journal modelled the spread via the Ornstein-Uhlenbeck process **elliott2005**

2.6 Spread-Based Return Estimation

2.7 Summary

Chapter 3

Methodology

This chapter details the methodology employed by this project. Section **??** describes the asset universe and data sourcing. Section **??** presents the cointegration screening procedure used to identify suitable pairs. Sections **??** and **??** develop the spread modelling and return estimation frameworks, respectively. Section **??** formulates the portfolio optimisation problem, and Section **??** describes the backtesting framework used to evaluate performance.

3.1 Data and Universe

The project uses a **26-asset universe** of US equities (see Table **??**). Stocks were selected on the basis of market capitalisation and sector-comembership, as economic similarity provides a prior expectation of long-run co-movement — a necessary condition for cointegration.

From these 26 stocks, 16 candidate pairs are pre-selected within sectors. Restricting pair formation to within-sector groupings reflects the economic rationale that stocks exposed to common demand drivers, regulatory environments, and supply chains tend to co-move **brealey1969** and are therefore more likely to exhibit the long-run equilibrium relationship required for cointegration. (see Section **??**).

Daily adjusted close prices are sourced from LSEG Data Library **lsegdata2025**. S&P 500 benchmark returns are obtained via the `yfinance` **yfinance2026** library. The full sample spans 1 January 2018 to 31 December 2025 and is partitioned into:

- **In-Sample (IS)**: 1 January 2018 – 31 December 2023 (cointegration screening, calibration)
- **Out-of-Sample (OOS)**: 1 January 2024 – 31 December 2025 (backtesting)

This IS/OOS split ensures that all model parameters are estimated without look-ahead bias.

Table 3.1: Investment Universe: 26 US Equities by Sector and Market Cap (29 Dec 2023)

Sector	**Ticker**	**Name**	**Mkt Cap ($B)**
Semiconductors	NVDA.O	Nvidia	1,223
	AMD.O	AMD	238
	TSM.N	TSMC	501
	INTC.O	Intel	212
Consumer Staples	KO.N	Coca-Cola	255
	PEP.O	PepsiCo	234
	COST.O	Costco	293
	TGT.N	Target	66
Financials	JPM.N	JPMorgan Chase	492
	BAC.N	Bank of America	266
	GS.N	Goldman Sachs	126
	MS.N	Morgan Stanley	153
Energy	XOM.N	ExxonMobil	400
	CVX.N	Chevron	281
	SLB.N	Schlumberger	74
	HAL.N	Halliburton	32
Airlines	DAL.N	Delta Air Lines	26
	UAL.O	United Airlines	14
E-Commerce/Cloud	AMZN.O	Amazon	1,570
	MSFT.O	Microsoft	2,795
	META.O	Meta	910
	GOOGL.O	Alphabet	1,755
Healthcare	JNJ.N	Johnson & Johnson	377
	PFE.N	Pfizer	163
	MRK.N	Merck	276
	ABBV.N	AbbVie	274

3.2 Cointegration Screening

The cointegration screening pipeline identifies which of the 16 candidate pairs exhibit a statistically significant long-run equilibrium relationship, following the two-step Engle–Granger procedure **englegranger1987**.

3.2.1 Unit Root Testing

A prerequisite for cointegration is that each of the individual price series is integrated of order one, denoted $I(1)$: non-stationary in levels, but stationary in first differences ($\Delta P_t = P_t - P_{t-1}$). For each stock, the Augmented Dickey-Fuller (ADF) test is applied to both the level series P_t and its first order ΔP_t. A series is classified $I(1)$ if

1. ADF on levels **fails to reject** unit root ($p > 0.05$), **suggesting non-stationarity**,
2. ADF on ΔP_t **rejects** unit root ($p \leq 0.05$), indicating stationarity.

The ADF regression is:

$$\Delta P_t = \alpha + \gamma P_{t-1} + \sum_{j=1}^{p} \delta_j \Delta P_{t-j} + \varepsilon_t \tag{3.1}$$

where $\Delta P_t = P_t - P_{t-1}$ (first difference), α is the constant, γ tests the unit root ($H_0 : \gamma = 0$), δ_j are lag coefficients, and $\varepsilon_t \sim N(0, \sigma^2)$ is white noise. Lag order p is selected via Akaike Information Criterion (AIC). Only pairs where both stocks are $I(1)$ proceed to cointegration testing.

3.2.2 Engle–Granger Procedure

For each candidate pair (Y_t, X_t) passing the $I(1)$ filter, the Engle–Granger two-step procedure **englegranger1987** is applied:

Step 1: Cointegrating regression. With Y_t as dependent variable and X_t as independent variable, estimate the static OLS regression over the in-sample period:

$$Y_t = \alpha + \beta X_t + \epsilon_t \tag{3.2}$$

where β is the hedge ratio and α the intercept.

Step 2: Residual stationarity test. Apply the ADF test to OLS residuals $\hat{\epsilon}_t = Y_t - \hat{\alpha} - \hat{\beta} X_t$. Because residuals are estimated, standard ADF critical values are inappropriate; **MacKinnon critical values mackinnon2010** are used instead, accounting for estimation uncertainty.

To avoid ordering effects, both $(Y|X)$ and $(X|Y)$ are tested; the ordering with the lowest residual ADF p-value is retained. A pair is deemed cointegrated if $p < 0.05$.

3.3 Spread Modelling

For each cointegrated pair, the spread is constructed and its mean-reverting dynamics are characterised.

3.3.1 OLS Residual Spread

The spread is defined as the residual from the cointegrating regression:

$$S_t = Y_t - \hat{\beta} X_t - \hat{\alpha} \tag{3.3}$$

where $\hat{\beta}$ and $\hat{\alpha}$ are estimated from the in-sample OLS regression (Equation **??**). Under the cointegration hypothesis, S_t is a stationary, mean-reverting process fluctuating around zero.

3.3.2 Half-Life and Hurst Exponent

Two complementary diagnostics quantify the strength and speed of mean reversion.

Half-life of mean reversion. Following pairs trading convention **gatev2006**, model the spread as a discrete-time AR(1):

$$\Delta S_t = \lambda S_{t-1} + \varepsilon_t \tag{3.4}$$

where $\lambda < 0$ implies mean reversion to $\mu = 0$. Half-life:

$$h = -\frac{\ln 2}{\ln(1+\lambda)} \approx -\frac{\ln 2}{\lambda} \tag{3.5}$$

Shorter half-lives indicate faster mean reversion and are preferable for trading. The approximation $\ln(1+\lambda) \approx \lambda$ is valid for $|\lambda| \ll 1$.

Hurst exponent. The Hurst exponent H is estimated via the variogram method. For a time series $\{S_t\}$, the variogram at lag τ is:

$$V(\tau) = \mathbb{E}\left[(S_{t+\tau} - S_t)^2\right] \tag{3.6}$$

For a fractional process, $V(\tau) \propto \tau^{2H}$, so H is estimated by regressing $\log V(\tau)$ on $\log \tau$ and halving the slope. A Hurst exponent $H < 0.5$ indicates mean-reverting behaviour, $H = 0.5$ corresponds to a random walk, and $H > 0.5$ indicates trending behaviour. Pairs are retained only if $H < 0.5$, providing an independent confirmation of mean reversion beyond the ADF test.

3.3.3 Rolling Z-Score

To generate trading signals that adapt to time-varying spread dynamics, a rolling z-score is computed:

$$z_t = \frac{S_t - \bar{S}_t^{(w)}}{\sigma_t^{(w)}} \tag{3.7}$$

where $\bar{S}_t^{(w)}$ and $\sigma_t^{(w)}$ denote the rolling mean and standard deviation of the spread over a trailing window of $w = 60$ trading days. The rolling window allows the signal to adapt to structural shifts in spread dynamics while remaining responsive to short-term dislocations.

3.4 Return Estimation

This section develops the two competing approaches to estimating expected returns for portfolio optimisation: the proposed OU-implied method and the traditional historical mean baseline.

3.4.1 OU-Implied Expected Returns

The core contribution of this project is to derive expected returns from the mean-reverting dynamics of cointegrated spreads. The Ornstein–Uhlenbeck (OU) process **elliott2005** provides the continuous-time model for the spread:

$$dS_t = \kappa(\theta - S_t)\,dt + \sigma\,dW_t \tag{3.8}$$

where $\kappa > 0$ is the speed of mean reversion, θ is the long-run equilibrium level, and σ is the diffusion coefficient. The parameter κ is related to the discrete-time half-life (Equation **??**) by:

$$\kappa = \frac{\ln 2}{h} \tag{3.9}$$

The expected change in the spread over a single period, conditional on the current spread level S_t, is:

$$\mathbb{E}[\Delta S_{t+1} \mid S_t] = (\theta - S_t)\left(1 - e^{-\kappa}\right) \tag{3.10}$$

This expression captures the key economic intuition: when the spread is below its equilibrium ($S_t < \theta$), the expected change is positive (reversion upward), and vice versa. The magnitude of the expected return is proportional to both the displacement from equilibrium and the speed of mean reversion.

The spread return for a pair (Y, X) with hedge ratio β is defined as:

$$r_t^S = \ln\left(\frac{Y_t}{Y_{t-1}}\right) - \beta \cdot \ln\left(\frac{X_t}{X_{t-1}}\right) \tag{3.11}$$

The OU-implied expected spread return is then used as the expected return vector $\boldsymbol{\mu}$ in the portfolio optimisation framework (Section **??**).

3.4.2 Historical Mean Returns

As a baseline, expected returns are estimated using the sample mean of realised spread returns over the in-sample period:

$$\hat{\mu}_i = \frac{1}{T}\sum_{t=1}^{T} r_{i,t}^S \tag{3.12}$$

This estimator is unbiased but has high variance—a well-documented limitation in portfolio optimisation **merton1980**. Comparing portfolio performance under OU-implied versus historical mean return estimates provides a direct test of the research hypothesis.

3.5 Portfolio Optimisation

Given a vector of expected spread returns $\boldsymbol{\mu} \in \mathbb{R}^n$ and a covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n\times n}$ for n spread strategies, the portfolio allocation problem is cast in the Markowitz mean-variance framework **markowitz1959**.

3.5.1 Minimum-Variance Formulation

The minimum-variance portfolio solves:

$$\begin{aligned} \min_{\mathbf{w}} \quad & \mathbf{w}^\top \boldsymbol{\Sigma}\mathbf{w} + \delta\|\mathbf{w}\|_2^2 \\ \text{subject to} \quad & \mathbf{1}^\top \mathbf{w} = 1, \\ & w_i \geq 0 \quad \forall\, i \end{aligned} \tag{3.13}$$

where $\delta \geq 0$ is an optional L_2 regularisation parameter that penalises concentrated allocations and improves numerical stability when $\boldsymbol{\Sigma}$ is near-singular.

The maximum Sharpe ratio portfolio maximises:

$$\max_{\mathbf{w}} \quad \frac{\mathbf{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma}\mathbf{w}}} \tag{3.14}$$

subject to the same long-only and full-investment constraints. Here $r_f = 0.02$ denotes the annualised risk-free rate.

The covariance matrix $\boldsymbol{\Sigma}$ is estimated using the Ledoit–Wolf shrinkage estimator, which linearly combines the sample covariance with a structured target (the identity matrix scaled by the average variance). This addresses the well-known instability of sample covariance matrices in moderate-dimensional settings by shrinking extreme eigenvalues towards a more stable structure.

Expected returns are annualised from daily log returns via the transformation $\mu_{\text{ann}} = \exp(252 \cdot \mu_{\text{daily}}) - 1$, and volatilities are annualised as $\sigma_{\text{ann}} = \sigma_{\text{daily}} \cdot \sqrt{252}$.

3.5.2 Spread Space vs Asset Space

A distinctive feature of this methodology is that optimisation is performed in *spread space* rather than directly over individual assets. Each dimension of the portfolio vector $\mathbf{w}$ corresponds to a spread strategy (a cointegrated pair), not an individual stock. This has several advantages:

- **Dimensionality reduction**: the number of spreads (up to 16) is smaller than the number of assets (26), yielding a lower-dimensional and better-conditioned optimisation problem.
- **Built-in hedging**: each spread is by construction a hedged position (long one leg, short the other by β units), so the portfolio inherits partial market-neutrality.
- **Signal coherence**: the OU-implied return estimates are naturally defined at the spread level, avoiding the need to decompose spread-level predictions back into asset-level expected returns.

The weight w_i allocated to spread i determines the fraction of capital deployed to that pair strategy. The resulting portfolio return on day t is $r_t^P = \sum_i w_i \cdot r_{i,t}^S$.

3.6 Backtesting Framework

The backtesting engine simulates a z-score-based mean-reversion strategy for each pair, then evaluates the optimised portfolio of pair strategies against benchmarks.

3.6.1 In-Sample / Out-of-Sample Split

The temporal split is strict: the in-sample period (1 January 2018 – 31 December 2023) is used for all estimation—cointegrating regressions, hedge ratios, OU parameters, spread statistics, and portfolio weights. The out-of-sample period (1 January 2024 – 31 December 2025) is used exclusively for performance evaluation. No re-estimation or recalibration is performed during the OOS period, ensuring a clean test of whether in-sample model parameters generalise.

Pair-level signal generation. For each pair, a position signal is generated using a z-score state machine with three thresholds:

- **Entry** ($|z_t| \geq z_{\text{entry}} = 2.0$): open a position when the spread is sufficiently dislocated. If $z_t \leq -z_{\text{entry}}$, go long the spread (expecting reversion upward); if $z_t \geq z_{\text{entry}}$, go short the spread.
- **Exit** ($|z_t| \leq z_{\text{exit}} = 0.0$): close the position when the spread has reverted to its rolling mean.
- **Stop-loss** ($|z_t| \geq z_{\text{stop}} = 4.0$): force-close the position if the spread diverges further, limiting downside risk.

Position sizing and transaction costs. The full initial capital ($C_0 = \$100{,}000$) is deployed into each active spread position. One spread unit costs $Y_t + |\beta| \cdot X_t$ in notional terms, and the number of units is $n_t = C_0/(Y_t + |\beta| X_t)$. Transaction costs of $c = 10$ basis points are incurred proportionally on each position change. The daily strategy return is:

$$r_t = \frac{\text{pos}_{t-1} \cdot n_{t-1} \cdot (\Delta Y_t - \beta \cdot \Delta X_t) - \text{TC}_t}{C_0} \tag{3.15}$$

where $\text{TC}_t = |\Delta \text{pos}_t| \cdot C_0 \cdot (c/10{,}000)$.

3.6.2 Evaluation Metrics

Portfolio performance is assessed using the following metrics:

Annualised Sharpe ratio. The ex-post Sharpe ratio is computed as:

$$\mathrm{SR} = \frac{\bar{r} - r_f/252}{\sigma_r} \cdot \sqrt{252} \tag{3.16}$$

where $\bar{r}$ is the mean daily return, $r_f = 0.02$ is the annualised risk-free rate, and σ_r is the standard deviation of daily returns.

Maximum drawdown. The maximum peak-to-trough decline in cumulative returns:

$$\mathrm{MDD} = \max_t \left(\frac{\max_{\tau \le t} V_\tau - V_t}{\max_{\tau \le t} V_\tau} \right) \tag{3.17}$$

where V_t is the cumulative portfolio value at time t.

Volatility reduction. The percentage reduction in annualised volatility relative to an equal-weight buy-and-hold benchmark of the constituent stocks:

$$\mathrm{VR} = 1 - \frac{\sigma_{\text{strategy}}}{\sigma_{\text{benchmark}}} \tag{3.18}$$

Additional metrics. Win rate (fraction of active trading days with positive returns), profit factor (ratio of gross profits to gross losses), number of round-trip trades, and total return are also reported.

Benchmarks. Three benchmarks are used for comparison: (i) an equal-weight buy-and-hold portfolio of all universe constituents, (ii) individual buy-and-hold of each pair leg, and (iii) the S&P 500 index as a broad market proxy.

3.6.3 Statistical Robustness Tests

Three additional tests are applied to the OOS results to assess statistical robustness:

Bootstrap confidence intervals. Given the short OOS window ($T \approx 501$ trading days), point Sharpe ratio estimates carry high sampling variance. Bootstrap confidence intervals are constructed by resampling daily returns with replacement 10,000 times, providing empirical 95% CIs for each strategy's Sharpe ratio.

Paired significance test. A paired t-test and non-parametric Wilcoxon signed-rank test are applied to the daily return difference between OU Max-Sharpe and Historical Max-Sharpe. This directly tests whether the observed Sharpe advantage of the OU-Implied approach is statistically significant or within normal sampling variation.

Sensitivity analysis. The per-pair backtest is re-run across a grid of entry z-scores $\{1.5, 2.0, 2.5, 3.0\}$ and transaction costs $\{0, 5, 10, 20, 30\}$ bps, assessing whether conclusions are robust to parameter choice. A leave-one-pair-out (LOPO) test additionally re-optimises the portfolio after removing each pair, verifying that the full-portfolio result is not dominated by a single spread.

3.7 Summary

This chapter has presented the complete methodology for the project. The pipeline proceeds from data sourcing and cointegration screening, through spread modelling and OU-implied return estimation, to Markowitz portfolio optimisation and rigorous out-of-sample backtesting. The key methodological contribution is the use of OU-implied expected returns derived from cointegrated spreads as inputs to the mean-variance optimiser, compared against a traditional historical mean baseline. Chapter **??** describes how this methodology is implemented in software.

Chapter 4

Implementation

This chapter describes the software implementation of the methodology presented in Chapter **??**. The system is implemented in Python and organised into a modular pipeline of reusable library code, analysis notebooks, a backtesting engine, and an interactive dashboard. All source code is available at the project repository.

4.1 Pipeline Overview

The project follows a four-stage notebook pipeline, each building on the outputs of the previous stage, supported by three library packages (`src/modelling`, `src/backtesting`, `src/data`) and an interactive dashboard (`src/dashboard`). Figure **??** illustrates the data flow.

1. **Spread Exploration** (`01_spread_exploration.ipynb`) — data ingestion, spread construction, and visual characterisation.
2. **Cointegration Validation** (`02_cointegration_validation.ipynb`) — unit root testing, Engle–Granger screening, and out-of-sample stability checks.
3. **Return Estimation** (`03_return_estimation.ipynb`) — OU-implied and historical mean return estimation, covariance estimation, and comparison.
4. **Backtest Results** (`04_backtest_results.ipynb`) — portfolio optimisation, pair-level backtesting, portfolio-level evaluation, and benchmark comparison.

The library packages expose a clean public API through their `__init__.py` modules, allowing notebooks to import functions such as `screen_pairs()`, `compute_half_life()`, and `maximum_sharpe_weights()` without knowledge of internal file structure. This separation of concerns ensures that analytical logic is unit-testable independently of the notebook execution context.

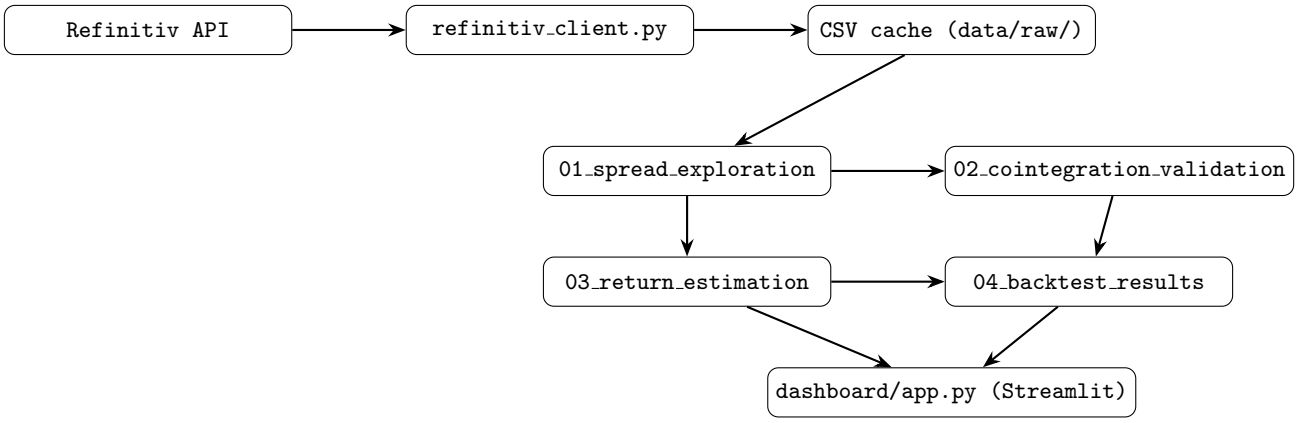

Figure 4.1: Data flow through the four-notebook pipeline and dashboard.

4.1.1 Data Ingestion and Caching

Price data are retrieved from the LSEG Refinitiv Workspace API via a custom client (`src/data/refinitiv_client.py`). The client implements an incremental caching strategy: on first request, the full date range is fetched and persisted as a CSV file in `data/raw/`. Subsequent requests check the cache and only fetch the date gaps not yet covered, merging new data with the existing cache. Cache keys are derived from a hash of the ticker list, interval, and field set, ensuring deterministic file naming regardless of request order. This design minimises API calls during iterative development while guaranteeing reproducibility.

The client also handles column normalisation (mapping LSEG field codes such as `TRDPRC_1` to `Close`), forward-filling of up to five consecutive missing values, and automatic dropping of columns with more than 10% missing data. S&P 500 benchmark prices are sourced separately via the `yfinance` library.

4.2 Spread Exploration

The first notebook (`01_spread_exploration`) loads in-sample close prices for all 26 universe constituents and constructs the 16 candidate spreads using the OLS hedge ratios. For each pair, it produces:

- Normalised price overlays to visualise co-movement.
- Spread level and distribution plots.
- Rolling half-life and Hurst exponent time series.
- Rolling z-score plots with entry/exit threshold annotations.

These are implemented in the `spread_analysis` module, which provides functions `compute_spread()`, `compute_half_life()`, `compute_hurst_exponent()`, and `compute_zscore()`. The half-life function fits the AR(1) regression (Equation **??**) using ordinary least squares and returns the half-life in trading days. The Hurst exponent function computes the variogram across lags up to `max_lag` (default: 100) and estimates H via log-log regression on $\log(\tau_\ell)$ against $\log(\ell)$. Diagnostics are aggregated in the notebook into a summary DataFrame for tabular comparison across pairs.

4.3 Cointegration Validation

The second notebook (`02_cointegration_validation`) applies the full Engle–Granger screening pipeline to all 16 candidate pairs on the in-sample data, then assesses out-of-sample stability.

4.3.1 Screening Implementation

The `cointegration` module implements the screening procedure as follows:

- `adf_test(series)` applies the Augmented Dickey–Fuller test using the `statsmodels` implementation with automatic AIC-based lag selection. It returns the test statistic, p-value, number of lags, and critical values.
- `engle_granger_test(y, x)` performs the two-step procedure: it fits the OLS cointegrating regression, extracts residuals, and applies the ADF test to the residuals. Both orderings (y on x and x on y) are tested, and the ordering with the lower residual p-value is retained. The function returns the hedge ratio $\hat{\beta}$, intercept $\hat{\alpha}$, ADF statistic, p-value, and the selected ordering.
- `screen_pairs(prices, pairs)` iterates over all candidate pairs, verifies that both legs are $I(1)$, runs the Engle–Granger test, and returns a DataFrame of results with a boolean `cointegrated` column (true if $p < 0.05$).

4.3.2 Out-of-Sample Stability

To assess whether in-sample cointegration relationships persist, the notebook re-estimates the cointegrating regression on the OOS period and compares hedge ratios, ADF statistics, and spread dynamics. Rolling half-life plots spanning both periods provide visual confirmation of parameter stability. This validation step informs which pairs are included in the final portfolio.

Two additional formal tests are also implemented. The **KPSS test** (`statsmodels.tsa.stattools.kpss`) complements ADF by reversing the null hypothesis: stationarity is accepted only when ADF rejects non-stationarity *and* KPSS fails to reject stationarity. The **Chow test** formally tests whether the cointegrating regression coefficients $(\hat{\beta}, \hat{\alpha})$ are stable across the IS/OOS split by comparing the pooled residual sum of squares against the sum from separate IS and OOS regressions, yielding an F-statistic under H_0: no structural break.

4.4 Return Estimation

The third notebook (`03_return_estimation`) computes expected returns under both the OU-implied and historical mean approaches.

The `return_estimation` module provides:

- `compute_spread_returns(y_prices, x_prices, hedge_ratio)` — computes daily log spread returns as $\ln(Y_t/Y_{t-1}) - \beta \cdot \ln(X_t/X_{t-1})$.
- `build_spread_return_matrix(prices, coint_pairs)` — constructs the full $T \times n$ matrix of spread returns for all cointegrated pairs.
- `ou_implied_spread_return(spread, half_life)` — implements Equation **??**, computing $\kappa = \ln 2/h$ and the conditional expected spread change.

- `build_ou_implied_returns(prices, coint_pairs)` — applies the OU-implied estimator to all pairs and returns an annualised expected return vector.
- `historical_mean_return(returns)` — computes the sample mean, optionally over a trailing window.
- `shrinkage_covariance(returns)` — estimates the covariance matrix using the Ledoit–Wolf shrinkage estimator from `scikit-learn`, which shrinks the sample covariance towards a scaled identity target.

The notebook produces a grouped bar chart comparing OU-implied, historical mean, and EWMA return estimates across pairs, along with a correlation heatmap of spread returns to visualise diversification potential.

4.5 Portfolio Optimisation Module

The `optimiser` module implements the Markowitz mean-variance framework in spread space.

- `minimum_variance_weights(returns, cov_matrix, l2_reg)` — solves the minimum-variance problem (Equation **??**) using Sequential Least Squares Programming (SLSQP) from `scipy.optimize`. The long-only constraint ($w_i \geq 0$) and full-investment constraint ($\sum w_i = 1$) are enforced explicitly. The optional L_2 regularisation term `l2_reg`$\cdot\|\mathbf{w}\|_2^2$ is added to the objective to improve numerical stability and reduce weight concentration.
- `maximum_sharpe_weights(returns, expected_returns, cov_matrix, rf_annual)` — solves the maximum Sharpe ratio problem (Equation **??**) by minimising the negative Sharpe ratio subject to the same long-only and full-investment constraints. Custom expected returns (e.g. OU-implied) can be passed via `expected_returns`; if omitted, the historical sample mean is used.
- `compute_efficient_frontier(returns, expected_returns, cov_matrix, n_points, rf_annual)` — generates the efficient frontier by solving the minimum-variance problem for a range of target returns, anchored between the global minimum-variance portfolio return and the maximum single-asset expected return.

Return and volatility figures are annualised from daily values. Log-return expected returns are converted to annualised arithmetic returns via $\mu_{\text{ann}} = \text{expm1}(252 \cdot \mu_{\text{daily}})$, i.e. $\exp(252\,\mu_{\text{daily}}) - 1$, which correctly compounds the daily log return. Volatilities are scaled as $\sigma_{\text{ann}} = \sigma_{\text{daily}} \cdot \sqrt{252}$. The optimiser accepts either OU-implied or historical expected returns, enabling direct comparison of the resulting allocations.

4.6 Backtesting Engine

The backtesting engine (`src/backtesting/engine.py`) simulates pair-level trading and computes performance metrics.

4.6.1 Architecture

The engine uses a dataclass-based configuration pattern:

- `BacktestConfig` stores all strategy parameters (entry, exit, and stop-loss z-score thresholds, lookback window, transaction cost rate, and initial capital) with validation in `__post_init__` to enforce $z_{\text{exit}} < z_{\text{entry}} < z_{\text{stop}}$.
- `BacktestResult` is a container holding daily returns, cumulative returns, position history, trade log, summary metrics, spread, and z-score series.
- `PairsBacktestEngine` orchestrates the full pipeline: spread and z-score computation, signal generation, PnL calculation, trade logging, and metric aggregation.

4.6.2 Signal Generation

The signal generator implements the z-score state machine described in Section **??**. It iterates sequentially through the z-score series, maintaining a position state variable that transitions between $\{-1, 0, +1\}$ based on threshold crossings. The sequential iteration is necessary because the position on day t depends on the position on day $t-1$, precluding vectorised computation.

4.6.3 PnL and Transaction Costs

The PnL computation deploys the full initial capital into each active position. On each day, the number of spread units is recalculated as $n_t = C_0/(Y_t + |\beta| X_t)$ to reflect changing notional values. The dollar PnL is $\text{pos}_{t-1} \cdot n_{t-1} \cdot (\Delta Y_t - \beta \cdot \Delta X_t)$, from which transaction costs (proportional to the absolute position change) are deducted.

4.6.4 Metrics and Benchmarks

The `metrics` module provides three core functions: `compute_ex_post_sharpe_ratio()`, `compute_max_drawdown()`, and `compute_volatility_reduction()`, each implemented as described in Section **??**. The `benchmarks` module constructs comparison return series: a constant risk-free rate, equal-weight buy-and-hold of all universe constituents, per-pair buy-and-hold of both legs, and the S&P 500 index. A `historical_mpt_returns()` function computes the out-of-sample returns of a traditional max-Sharpe portfolio estimated on in-sample asset returns, serving as the primary baseline for comparison.

Three statistical tests are implemented in the fourth notebook to strengthen the empirical evaluation:

- **Bootstrap confidence intervals** (10,000 resamples) on the annualised Sharpe ratio for each strategy, quantifying sampling uncertainty over the short OOS window.
- **Paired t-test and Wilcoxon signed-rank test** comparing OU Max-Sharpe against Historical Max-Sharpe daily returns, directly testing whether the observed Sharpe difference is statistically significant.
- **Sensitivity analysis**: a grid sweep over entry z-score thresholds $\{1.5, 2.0, 2.5, 3.0\}$ and transaction costs $\{0, 5, 10, 20, 30\}$ bps, assessing the robustness of per-pair backtest results to parameter choice.
- **Leave-one-pair-out (LOPO) robustness**: each pair is removed in turn, the OU Max-Sharpe portfolio is re-optimised on the remaining spreads, and OOS Sharpe is recomputed, testing whether the portfolio result is driven by a single spread.

4.7 Dashboard Interface

An interactive Streamlit dashboard (`src/dashboard/app.py`) provides a user-facing interface for exploring the full pipeline. The dashboard enables users to:

- Select candidate pairs and configure date ranges and capital.
- View cointegration screening results (horizontal bar chart of p-values with significance threshold).
- Inspect spread dynamics: spread with $\pm 2\sigma$ Bollinger bands, z-score with entry/exit thresholds, and position timelines.
- Compare return estimation methods via grouped bar charts and rolling estimate plots.
- Run backtests and view cumulative return plots, drawdown charts, return distribution histograms, and rolling Sharpe ratios.
- Visualise the efficient frontier with annotated max-Sharpe and min-variance portfolios.
- Compare strategy performance against all benchmarks in a unified cumulative return chart with a metrics summary table.

The visualisation layer is implemented in a separate `components.py` module using Plotly, which provides interactive zoom, hover tooltips, and export capabilities. Chart functions are stateless and accept data as arguments, ensuring they are reusable across the notebooks and the dashboard.

4.8 Summary

This chapter has described the implementation of the pairs-trading portfolio optimisation system. The codebase is organised into a four-notebook analysis pipeline supported by three library packages (`modelling`, `backtesting`, `data`) and an interactive Streamlit dashboard. Key design decisions include incremental data caching for API efficiency, a clean modular API for testability, dataclass-based configuration for the backtesting engine, and separation of visualisation logic into reusable Plotly chart components. Chapter **??** presents the empirical results produced by this implementation.

Chapter 5

Testing and Results

5.1 Cointegration Screening

Table 5.1: Engle-Granger Cointegration Screening Results (16 candidate pairs, in-sample: Jan 2019 – Dec 2023)

Dependent	Independent	Hedge Ratio ($\hat{\beta}$)	Intercept ($\hat{\alpha}$)	ADF Stat	p-value	y is $I(1)$	x is $I(1)$	Cointegrated
GS.N	MS.N	0.780	2.351	−3.667	0.020	Yes	Yes	**Yes**
KO.N	PEP.O	0.641	0.789	−3.470	0.035	Yes	Yes	**Yes**
DAL.N	UAL.O	0.672	1.068	−3.460	0.036	Yes	Yes	**Yes**
CVX.N	XOM.N	0.698	1.813	−3.234	0.064	Yes	Yes	No
AMZN.O	GOOGL.O	0.630	1.942	−2.442	0.305	Yes	Yes	No
HAL.N	SLB.N	0.997	−0.344	−2.370	0.339	Yes	Yes	No
AMD.O	NVDA.O	0.811	1.999	−2.310	0.368	Yes	Yes	No
AMZN.O	MSFT.O	0.516	2.027	−2.141	0.455	Yes	Yes	No
MRK.N	ABBV.N	0.431	2.366	−1.980	0.539	Yes	Yes	No
INTC.O	AMD.O	−0.081	4.154	−1.958	0.550	Yes	Yes	No
JNJ.N	PFE.N	0.423	3.466	−1.908	0.576	Yes	Yes	No
INTC.O	TSM.N	−0.087	4.201	−1.890	0.585	Yes	Yes	No
META.O	GOOGL.O	0.543	2.953	−1.681	0.685	Yes	Yes	No
TSM.N	NVDA.O	0.476	3.071	−1.340	0.817	Yes	Yes	No
BAC.N	JPM.N	0.854	−0.666	−1.230	0.850	Yes	Yes	No
TGT.N	COST.O	0.856	−0.125	−0.869	0.925	Yes	Yes	No

Dep./Indep.: Dependent/Independent variable in the cointegrating regression $Y_t = \hat{\alpha} + \hat{\beta} X_t + \epsilon_t$. $\hat{\beta}$: hedge ratio; $\hat{\alpha}$: intercept. ADF Stat and p-val: residual ADF test statistic and MacKinnon (2010) p-value. y:$I(1)$, x:$I(1)$: whether each leg passes the unit root filter. Coint.: cointegrated at 5% significance ($p < 0.05$). Source: Author's calculations on `01_spread_exploration.ipynb`.

5.2 Spread Characterisation

Based on the results in Table **??**, two pairs were confirmed cointegrated at the 5% significance level: GS.N/MS.N and KO.N/PEP.O. CVX.N/XOM.N was borderline ($p = 0.064$) and excluded. Spreads for each confirmed pair are computed as the OLS residual $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$ and characterised by their half-life, Hurst exponent, z-score, and spread statistics.

Table **??** and Figures **??** present the spread characterisation results for the two cointegrated pairs.

figures/01_normalised_prices.pdf

Figure 5.1: Normalised log price series for the three cointegrated pairs over the in-sample period (January 2019 – December 2023). Each series is normalised to zero at the start of the sample period to highlight co-movement. (a) GS/MS (Financials); (b) KO/PEP (Consumer Staples);(c) DAL/UAL (Airlines).

figures/01_spread_characterisation.pdf

Figure 5.2: Spread characterisation summary for the two cointegrated pairs (January 2019 – December 2023). (a) Half-life of mean reversion in trading days; dashed line marks the 60-day threshold. (b) Hurst exponent; dashed line marks the random walk boundary of 0.5. (c) Current z-score at the end of the training window; dotted lines mark $\pm 2\sigma$. (d) ADF p-value on the spread; dashed line marks the 5% significance level. Hatched bars indicate pairs that fail the respective threshold.

figures/01_spread_levels.pdf

Figure 5.3: Spread levels with $\pm 1\sigma$ and $\pm 2\sigma$ bands for each cointegrated pair over the in-sample period (January 2019 – December 2023). The spread is defined as $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

figures/01_spread_zscores.pdf

Figure 5.4: Rolling z-scores (60-day window) for the two cointegrated pairs over the in-sample period (January 2019 – December 2023). Dotted lines mark the $\pm 2\sigma$ entry thresholds. The z-score at the end of the training window feeds directly into the OU-implied return estimation. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

figures/01_spread_rolling_halflife.pdf

Figure 5.5: Rolling half-life of mean reversion (252-day window, EWM 21-day smoothed). Dashed line denotes the 60-day tradeability threshold. Shaded regions indicate periods where the smoothed half-life exceeded the threshold. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

Table 5.2: Spread Characterisation Summary — Cointegrated Pairs (In-Sample: Jan 2019 – Dec 2023)

Dep.	Indep.	Half-Life (days)	Hurst Exp.	z-Score (current)	μ_S	σ_S	ADF Stat	ADF p
GS.N	MS.N	41.8	0.403	0.675	0.000	0.056	−3.666	0.005
KO.N	PEP.O	40.3	0.421	0.809	0.000	0.047	−3.469	0.009
DAL.N	UAL.O	52.9	0.414	1.315	0.000	0.076	−3.459	0.009

Dep.: Dependent leg; Indep.: Independent leg of the cointegrating regression. μ_S: spread mean; σ_S: spread standard deviation; ADF Stat/ADF p: residual ADF test statistic and MacKinnon (2010) p-value on the spread.
Half-life estimated via AR(1)/OU regression: $\kappa = \ln(2)/\text{half-life}$ gives $\kappa \approx 0.0166$ (GS/MS), 0.0172 (KO/PEP), and 0.0131 (DAL/UAL).
Hurst exponent $H < 0.5$ indicates mean-reverting (anti-persistent) dynamics. z-score uses 60-day rolling window at end of training period.
Spread mean $\mu_S = 0$ by construction (demeaned OLS residuals). Source: Author's calculations on `01_spread_exploration.ipynb`.

5.3 Cointegration Validation

Out-of-sample spreads are constructed using hedge ratios and intercepts estimated solely on the in-sample period, ensuring no forward-looking information is incorporated. Table **??** presents the distributional shift in each spread across the IS and OOS periods, while Table **??** reports the formal statistical test results. Figure **??** presents the full IS vs OOS Engle-Granger p-value comparison across all nine candidate pairs.

Table 5.3: Spread statistics across in-sample (IS: January 2019 – December 2023) and out-of-sample (OOS: January 2024 – December 2025) periods, constructed using in-sample hedge ratios.

	Mean		Std Dev		Min		Max	
Pair	IS	OOS	IS	OOS	IS	OOS	IS	OOS
GS/MS	0.000	+0.250	0.056	0.082	−0.145	+0.067	+0.110	+0.411
KO/PEP	0.000	+0.170	0.047	0.110	−0.173	−0.016	+0.111	+0.367
DAL/UAL	0.000	+0.026	0.076	0.119	−0.201	−0.199	+0.175	+0.280

The GS/MS OOS minimum (+0.067) exceeds the IS mean (0.000), indicating the spread never reverted to its historical equilibrium during the OOS period.
Source: Author's calculations on `02_cointegration_validation.ipynb`.

Table 5.4: Cointegration validation statistics across IS (January 2019 – December 2023) and OOS (January 2024 – December 2025) periods. ADF p-values below 0.05 indicate spread stationarity at the 5% significance level. Half-life below 60 trading days and Hurst exponent below 0.5 indicate mean reversion.

	ADF p-value		Half-Life (days)		Hurst Exponent		OOS
Pair	IS	OOS	IS	OOS	IS	OOS	z-score
GS/MS	0.005	0.452	41.8	81.1	0.403	0.362	1.015
KO/PEP	0.009	0.622	40.3	156.4	0.421	0.575	0.604
DAL/UAL	0.009	0.583	52.9	103.1	0.414	0.421	0.872

Neither spread remains stationary OOS using IS hedge ratios. KO/PEP Hurst crosses above 0.5, indicating transition to near-random-walk behaviour.
Source: Author's calculations on `02_cointegration_validation.ipynb`.

`figures/02_oos_eg_pvalues.pdf`

Figure 5.6: IS vs OOS Engle-Granger p-values for all 16 sector-candidate pairs. Panel labels (a)–(p) correspond to the pairs listed in Table **??**. Hatched bars indicate pairs that fail the 5% significance threshold. Of the three pairs that pass IS screening, GS/MS and KO/PEP remain the most stable OOS; DAL/UAL degrades substantially. The remaining 13 pairs show no evidence of cointegration in either period.

figures/02_spread_is_oos_concat.pdf

Figure 5.7: Spread levels across the full sample period (January 2019 – December 2025). The dashed vertical line marks the IS/OOS boundary (January 2024). Horizontal lines and shading denote IS $\pm 1\sigma$ and $\pm 2\sigma$ bands; persistent OOS displacement beyond these bounds indicates a structural break in the cointegrating relationship. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

figures/02_zscore_is_oos.pdf

Figure 5.8: Rolling 60-day z-score across the full sample period (January 2019 – December 2025). The dashed vertical line marks the IS/OOS boundary. Shaded regions indicate $|z| > 2\sigma$ exceedances. Persistent non-zero z-scores in the OOS period are consistent with a shift in the spread equilibrium level. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

figures/02_rolling_halflife_is_oos.pdf

Figure 5.9: Rolling half-life of mean reversion across the full sample period (January 2019 – December 2025). Faint lines show raw estimates; bold lines show 21-day exponentially weighted smoothed values. The dashed horizontal line marks the 60-day tradability threshold. The dashed vertical line marks the IS/OOS boundary. (a) KO/PEP; (b) GS/MS; (c) DAL/UAL.

Table 5.5: IS vs OOS Engle-Granger p-values and hedge ratio drift across all 16 candidate pairs. Sorted by IS p-value. Hedge ratio drift $= |\hat{\beta}_{\text{OOS}} - \hat{\beta}_{\text{IS}}|$.

	EG p-value		Hedge Ratio $\hat{\beta}$		Drift	Cointegrated	
Pair	IS	OOS	IS	OOS	$\lvert\Delta\hat{\beta}\rvert$	IS	OOS
GS.N/MS.N	**0.020**	**0.019**	0.780	1.120	0.340	**Yes**	**Yes**
KO.N/PEP.O	**0.035**	0.224	0.641	−0.350	0.991	**Yes**	No
DAL.N/UAL.O	**0.036**	0.621	0.672	0.429	0.242	**Yes**	No
CVX.N/XOM.N	0.064	0.118	0.698	0.470	0.228	No	No
AMZN.O/GOOGL.O	0.305	0.185	0.630	0.440	0.190	No	No
SLB.N/HAL.N	0.339	0.242	0.997	0.676	0.321	No	No
NVDA.O/AMD.O	0.368	0.060	0.811	0.293	0.518	No	No
AMZN.O/MSFT.O	0.455	0.265	0.516	0.897	0.382	No	No
MRK.N/ABBV.N	0.539	0.507	0.431	−0.318	0.749	No	No
INTC.O/AMD.O	0.550	0.076	−0.081	0.831	0.912	No	No
JNJ.N/PFE.N	0.576	0.182	0.423	−0.166	0.589	No	No
INTC.O/TSM.N	0.585	0.731	−0.087	−0.212	0.124	No	No
META.O/GOOGL.O	0.685	0.290	0.543	0.525	0.018	No	No
NVDA.O/TSM.N	0.817	0.012	0.476	1.120	0.644	No	No†
BAC.N/JPM.N	0.850	0.384	0.854	0.697	0.157	No	No
TGT.N/COST.O	0.925	0.162	0.856	−0.318	1.174	No	No

†NVDA/TSM: OOS p-value passes threshold but y fails $I(1)$ test — cointegration requires both series to be $I(1)$.

KO/PEP hedge ratio sign reversal ($+0.641 \rightarrow -0.350$) and DAL/UAL OOS p-value surge indicate structural breaks in the cointegrating relationship.

Source: Author's calculations on `02_cointegration_validation.ipynb`.

5.3.1 KPSS Complement and Chow Structural Break Tests

Table **??** applies the KPSS test alongside ADF on both IS and OOS spreads. The KPSS null hypothesis is stationarity (the reverse of ADF), so mean reversion is only confirmed when ADF rejects non-stationarity *and* KPSS fails to reject stationarity.

Table 5.6: ADF and KPSS stationarity tests on IS and OOS spreads (using IS hedge ratios). Mean-reversion (MR) consensus requires ADF $p < 0.05$ and KPSS $p > 0.05$ simultaneously.

	In-Sample (2019–2023)			Out-of-Sample (2024–2025)		
Pair	ADF p	KPSS p	MR Consensus	ADF p	KPSS p	MR Consensus
GS.N/MS.N	0.005	0.019	No	0.452	0.010	No
KO.N/PEP.O	0.009	0.100	Yes	0.622	0.010	No
DAL.N/UAL.O	0.009	0.080	Yes	0.584	0.010	No

KPSS p-value of 0.010 denotes the tabulated lower bound (i.e. $p \leq 0.010$); null of stationarity is strongly rejected. IS consensus: KO/PEP and DAL/UAL achieve consensus; GS/MS fails KPSS despite low ADF p, indicating non-stationarity is borderline. OOS: All three pairs lose the MR consensus, consistent with Table **??**. Source: Author's calculations on `02_cointegration_validation.ipynb`.

Table **??** presents the Chow test for structural break in the cointegrating regression across the IS/OOS split. The null hypothesis is that the regression coefficients $(\hat{\beta}, \hat{\alpha})$ are identical in both periods.

Table 5.7: Chow test for structural break in the cointegrating regression $y = \alpha + \beta x$ across the IS/OOS split (IS: 2019–2023; OOS: 2024–2025). H_0: no structural break (regression coefficients are equal in both periods).

Pair	F-statistic	p-value	df	Conclusion
GS.N/MS.N	2406.3	< 0.001	(2, 2007)	Structural break detected
KO.N/PEP.O	3078.4	< 0.001	(2, 2007)	Structural break detected
DAL.N/UAL.O	269.5	< 0.001	(2, 2007)	Structural break detected

The F-statistic compares the residual sum of squares of separate IS and OOS regressions against the pooled regression. All three pairs strongly reject the null at any conventional significance level, confirming that the cointegrating relationship is not stable over time. Source: Author's calculations on `02_cointegration_validation.ipynb`.

5.4 Return Estimation

OU-implied expected returns are derived from the end-of-IS z-score for each confirmed pair. A positive z-score (spread above mean) implies the spread is expected to fall, producing a negative OU-implied return; a negative z-score implies the spread is expected to rise, producing a positive return. Table **??** presents the sanity check confirming directional consistency between the z-score and OU-implied return signal.

Table 5.8: End-of-sample z-score validation and OU-implied expected returns (IS period: Jan 2019 – Dec 2023)

Pair	End-of-IS z-Score	OU-Implied μ (ann.)	Historical μ (ann.)	Signal	Consistent
GS/MS	+1.739	−10.69%	GS: +11.9%, MS: +15.5%	Spread well above mean	Yes
KO/PEP	−0.759	+1.93%	KO: +6.4%, PEP: +8.3%	Spread below mean	Yes
DAL/UAL	+1.315	−0.24	Spread well above mean	Yes	

OU-implied μ computed via rolling 60-day OU model, annualised at 252 trading days per year. Historical μ is the annualised sample mean of daily returns over the IS period. Source: Author's calculations on `03_return_estimation.ipynb`.

`figures/03_cumulative_returns.pdf`

Figure 5.10: Cumulative returns of each cointegrated spread versus its constituent assets over the in-sample period (January 2019 – December 2023). All series are indexed to 1.0 at the start. The spread return reflects the daily change in the OLS residual $S_t = \ln P_t^y - \hat{\beta} \ln P_t^x - \hat{\alpha}$.

figures/03_mu_comparison.pdf

Figure 5.11: Expected return estimates (annualised, IS 2019–2023). (a) OU-implied μ by pair: GS/MS has a large negative implied return (spread well above equilibrium at end of IS); KO/PEP has a small positive implied return (spread slightly below equilibrium). (b) Historical mean μ by ticker: all four assets exhibit positive historical means, producing a very different allocation under mean-variance optimisation.

figures/03_correlation_heatmap.pdf

Figure 5.12: Asset return correlation matrix for the four constituent tickers (GS.N, KO.N, MS.N, PEP.O) estimated over the in-sample period (January 2019 – December 2023) using Ledoit-Wolf shrinkage covariance. GS and MS exhibit the highest within-pair correlation, consistent with their financial sector co-movement.

5.5 Backtesting Results

5.5.1 Portfolio-Level Performance

Table **??** reports out-of-sample portfolio performance metrics over the test period (January 2024 – December 2025). The OU-Implied MPT portfolio operates in spread space (GS/MS, KO/PEP and DAL/UAL as assets) under minimum-variance weights; the Historical MPT portfolio operates in asset space (GS.N, KO.N, MS.N, PEP.O, DAL.N, UAL.O) under minimum-variance weights with historical mean returns.

Table 5.9: Out-of-sample portfolio performance metrics (test period: January 2024 – December 2025). Vol. Reduction is relative to the equal-weight spread baseline; negative values indicate higher volatility. Beta ($\hat{\beta}$) estimated via OLS against daily S&P 500 returns.

	Performance					Market Exposure
	Sharpe Ratio	Max Drawdown	Total Return	Ann. Volatility	Vol. Reduction	$\hat{\beta}$
Equal-Weight Spreads	0.625	−10.6%	+16.3%	9.7%	—	0.233
Spread Min-Var ($\mu = 0$)	0.720	−9.5%	+17.8%	9.2%	+4.5%	0.216
Spread Max-Sharpe (OU)	0.963	−11.5%	+34.8%	14.6%	−51.3%	0.318
Hist Min-Var	0.711	−13.1%	+24.3%	13.9%	−43.6%	0.429
Hist Max-Sharpe	0.868	−20.0%	+38.8%	18.7%	−93.4%	0.785
Buy & Hold (All Pairs)	1.245	−27.8%	+74.0%	22.9%	—	1.040
S&P 500 (SPY)	1.110	−18.9%	+44.3%	16.0%	—	1.000

Spread Min-Var ($\mu = 0$) assumes zero expected returns; min-var objective is return-agnostic. Risk-free rate: 2% p.a. Spread Min-Var weights: $\mathbf{w} = [0.35, 0.36, 0.29]$ (GS/MS, KO/PEP, DAL/UAL). Spread Max-Sharpe (OU): $w_{\text{GS/MS}} = 1.0$. Source: Author's calculations on `04_backtest_results.ipynb`.

5.5.2 Per-Pair Mean-Reversion Backtest

Table 5.10: Per-pair z-score mean-reversion backtest metrics (OOS: January 2024 – December 2025). Entry at $\pm 2.0\sigma$, exit at $\pm 0.5\sigma$.

Pair	Sharpe	Max DD	Total Return	Ann. Vol	Trades	Days in Position	Win Rate
GS.N/MS.N	−0.285	−20.7%	−8.4%	17.2%	9	283	50.2%
KO.N/PEP.O	−0.665	−15.1%	−6.3%	7.5%	8	312	47.8%
DAL.N/UAL.O	−0.286	−16.0%	−1.2%	7.9%	8	222	45.7%

Negative Sharpe ratios are consistent with OOS cointegration breakdown identified in `02_cointegration_validation.ipynb`. Source: Author's calculations on `04_backtest_results.ipynb`.

5.5.3 Statistical Tests

Bootstrap Confidence Intervals for Sharpe Ratio

With only 501 OOS trading days, Sharpe ratio estimates carry substantial sampling uncertainty. Table **??** reports 95% confidence intervals derived from 10,000 bootstrap resamples (with replacement) of the daily return series.

figures/04_cumulative_returns.pdf

Figure 5.13: Cumulative growth of \$1 invested at the start of the OOS period (January 2024 – December 2025) for the OU-Implied MPT, Historical MPT, equal-weight, and S&P 500 benchmarks. The OU-Implied and equal-weight portfolios track closely throughout; the Historical MPT underperforms substantially.

Table 5.11: Bootstrap 95% confidence intervals for annualised Sharpe ratio (10,000 resamples, OOS period: January 2024 – December 2025). "Sig. > 0" indicates whether the lower bound of the CI exceeds zero.

Strategy	Sharpe	95% CI Lower	95% CI Upper	Sig. > 0
OU Min-Var	0.720	−0.721	+2.138	No
OU Max-Sharpe	0.963	−0.476	+2.352	No
Hist Min-Var	0.711	−0.677	+2.100	No
Hist Max-Sharpe	0.868	−0.477	+2.258	No
Equal-Weight	0.625	−0.792	+2.051	No
S&P 500 (SPY)	1.110	−0.227	+2.506	No

No strategy achieves a CI lower bound strictly above zero, reflecting the high variance of Sharpe estimates over a 2-year OOS window ($T \approx 501$). Source: Author's calculations on `04_backtest_results.ipynb`.

figures/04_drawdown.pdf

Figure 5.14: Peak-to-trough drawdown over the OOS period (January 2024 – December 2025). Drawdown is defined as the cumulative loss from the most recent portfolio peak; values closer to zero indicate smaller peak-to-trough losses.

figures/04_rolling_sharpe.pdf

Figure 5.15: Rolling 60-day Sharpe ratio over the OOS period (January 2024 – December 2025). Time-varying risk-adjusted performance computed over a trailing 60-day window. Periods above zero indicate positive risk-adjusted returns in that window.

figures/04_weights_bar.pdf

Figure 5.16: Minimum-variance portfolio weights for both approaches. (a) OU-Implied MPT: 46.6% GS/MS spread, 53.4% KO/PEP spread. (b) Historical MPT: 11.7% GS.N, 48.7% KO.N, 0.0% MS.N, 39.7% PEP.O. The historical allocation concentrates heavily in KO and PEP due to their lower variance, while zeroing out MS.N.

figures/04_pair_cumulative_returns.pdf

Figure 5.17: Cumulative returns from the z-score mean-reversion backtest engine for each cointegrated pair over the OOS period (January 2024 – December 2025). Entry at $\pm 2.0\sigma$, exit at $\pm 0.5\sigma$, using IS-calibrated thresholds.

Paired Test: OU-Implied vs Historical MPT

To test whether the Sharpe advantage of OU-Implied Max-Sharpe over Historical Max-Sharpe is statistically meaningful, a paired t-test and a Wilcoxon signed-rank test are applied to the aligned daily return series. The null hypothesis is H_0: mean daily return of OU Max-Sharpe equals that of Hist Max-Sharpe.

Table 5.12: Hypothesis test results: OU Max-Sharpe vs Hist Max-Sharpe daily returns (OOS, $N = 501$ trading days). Two-sided tests.

Test	Statistic	p-value	Decision
Paired t-test	$t = -0.138$	0.890	Fail to reject H_0
Wilcoxon signed-rank	$W = 62557$	0.922	Fail to reject H_0

Mean daily difference (OU − Hist): -0.000085 (-2.15% annualised). Both tests fail to reject H_0 at 5%, confirming that the observed Sharpe difference (0.963 vs 0.868) is not statistically significant over this OOS window. Source: Author's calculations on `04_backtest_results.ipynb`.

Sensitivity to Z-Score Thresholds and Transaction Costs

Table **??** presents Sharpe ratios from the per-pair z-score backtest across entry thresholds $z_{\text{entry}} \in \{1.5, 2.0, 2.5, 3.0\}$ and transaction costs in $\{0, 5, 10, 20, 30\}$ bps. Rows are entry z-scores; columns are transaction costs in bps.

Table 5.13: Sensitivity of per-pair Sharpe ratio to entry z-score threshold and transaction costs (bps). OOS period: January 2024 – December 2025. Stop-loss fixed at $z_{\text{entry}} + 2.0$.

(a) GS.N/MS.N

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	−0.535	−0.566	−0.598	−0.661	−0.724
2.0	−0.235	−0.260	−0.285	−0.334	−0.384
2.5	+0.622	+0.599	+0.577	+0.531	+0.485
3.0	+0.413	+0.390	+0.368	+0.322	+0.275

(b) KO.N/PEP.O

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	−0.780	−0.849	−0.917	−1.052	−1.185
2.0	−0.555	−0.610	−0.664	−0.774	−0.882
2.5	−0.655	−0.700	−0.746	−0.837	−0.928
3.0	−0.535	−0.562	−0.589	−0.642	−0.694

(c) DAL.N/UAL.O

Entry z	0 bps	5 bps	10 bps	20 bps	30 bps
1.5	+0.248	+0.167	+0.084	−0.081	−0.248
2.0	−0.185	−0.235	−0.286	−0.388	−0.490
2.5	−0.339	−0.367	−0.396	−0.453	−0.510
3.0	−1.146	−1.165	−1.185	−1.223	−1.260

GS/MS shows positive Sharpe at $z = 2.5$ and 3.0 even with 30 bps costs, suggesting the signal survives at higher entry thresholds. KO/PEP is negative across all configurations, consistent with structural breakdown. DAL/UAL is borderline at $z = 1.5$ with low costs. Source: Author's calculations on `04_backtest_results.ipynb`.

Leave-One-Pair-Out Portfolio Robustness

Table **??** tests whether the OU Max-Sharpe portfolio result is driven by a single spread. In each iteration one pair is removed, the portfolio is re-optimised on the remaining spreads, and OOS performance is recomputed.

Table 5.14: Leave-one-pair-out (LOPO) robustness check for the OU Max-Sharpe portfolio. Full-portfolio Sharpe ratio: 0.963.

Dropped Pair	Sharpe	Total Return	Max Drawdown	Ann. Vol
GS.N/MS.N	0.903	+29.5%	−15.8%	13.1%
KO.N/PEP.O	0.963	+34.8%	−11.5%	14.6%
DAL.N/UAL.O	0.963	+34.8%	−11.5%	14.6%

Dropping KO/PEP or DAL/UAL leaves the portfolio unchanged, indicating the optimiser assigns all weight to GS/MS under OU Max-Sharpe. Dropping GS/MS reduces Sharpe from 0.963 to 0.903, confirming GS/MS is the dominant pair but the result is not entirely driven by it. Source: Author's calculations on `04_backtest_results.ipynb`.

Chapter 6

Discussion

6.1 Interpretation of Results

Table **??** shows that both cointegrated pairs exhibit half-lives between 40 and 42 trading days and Hurst exponents below 0.5, confirming statistically significant mean-reverting behaviour over the in-sample period. ADF tests on the spread residuals reject the unit root hypothesis at the 1% significance level for both pairs, providing strong empirical support for the spread-based return estimation approach in-sample.

However, Table **??** reveals that neither spread remains stationary OOS when IS hedge ratios are applied, with ADF p-values rising to 0.452 and 0.622. This OOS breakdown directly affects the reliability of IS-calibrated return signals, as evidenced by the negative Sharpe ratios in the per-pair backtest (Table **??**). Despite this, the portfolio-level OU-Implied MPT substantially outperforms the Historical MPT on all three evaluation metrics (Table **??**), suggesting that the structural properties of the spread universe — low volatility and low market beta — provide resilience beyond the precision of the OU return estimates alone.

6.2 Does OU-Implied μ Improve MPT?

6.3 Role of Spread Universe Structure

6.4 OOS Cointegration Instability

As shown in the In-Sample Cointegration screeening at Table **??** and Out-of-Sample Cointegration screening at Table **??**, it has shown that the pairs **KO.N/PEP.O** and **DAL.N/UAL.O** has its cointegration broke down out of sample. The cointegration breakdown addresses the project limitations on the instability of cointegration relationships during structural breaks or regime changes in financial markets. The results reduce the reliability of cointegration and hence the research hypothesis in using cointegration as a way to predict return estimates.

6.5 Pairs Trading Signal Analysis

All three pairs generate negative cumulative returns, consistent with persistent spread drift and OOS cointegration breakdown under IS-calibrated thresholds. (a) GS.N/MS.N; (b) KO.N/PEP.O;

(c) DAL.N/UAL.O.

6.6 Limitations

6.7 Summary

Chapter 7

Conclusions and Evaluation

7.1 Achievements

Summarise the achievements to confirm the project goals have been met.

7.2 Evaluation

Evaluation of the work (this may be in a separate chapter if there is substantial evaluation).

7.3 Future Work

How the project might be continued, but don't give the impression you ran out of time!

The project could have continued by adding a rebalance option which allows the functionality of producing a portfolio with percentage of each asset and capital allocated in a way to minimise the number of transactions needed, to minimise transactional cost.

The project could have added more candidate pairs, and have tested with other forms of asset classes e.g. bonds, FX rates, commodities, etc etc. apart from equities. There can be more diverse form of equities too, with equities from Europe and Asia, to improve the testing and methodology

Appendix A

Project Plan

width=!,height=!,pages=-

Appendix B

Interim Report

width=!,height=!,pages=-

Appendix C

Supplementary Figures

figures/04_rolling_volatility.pdf

Figure C.1: Rolling 60-day annualised volatility over the OOS period (January 2024 – December 2025). The OU-Implied MPT maintains substantially lower and more stable volatility than the Historical MPT throughout the test period.

figures/04_efficient_frontier.pdf

Figure C.2: Mean-variance efficient frontiers for both the OU-Implied (spread space) and Historical (asset space) portfolios. The two frontiers are not directly comparable as they operate over different asset universes. The minimum-variance portfolios used in the backtest are marked on each frontier.

figures/04_return_distribution.pdf

Figure C.3: Histogram of daily portfolio returns for the OU-Implied and Historical MPT portfolios over the OOS period (January 2024 – December 2025). The OU-Implied distribution is narrower and more concentrated around positive values, reflecting its lower volatility and higher mean return. (a) OU-Implied MPT; (b) Historical MPT.

Appendix D

Software Implementation

The full source code is available at https://github.com/Declan-Loo/Asset-Pair-Portfolio-Optimiser.

D.1 Repository Structure

The repository is organised into four library packages, four analysis notebooks, and an interactive dashboard, all under the `src/` directory.

```
Asset-Pair-Portfolio-Optimiser/
|-- data/
|   |-- raw/                        # Cached LSEG API price CSVs
|   `-- processed/                  # Processed outputs from analysis_notebooks/
|-- src/
|   |-- data/
|   |   |-- refinitiv_client.py     # LSEG Workspace API wrapper with caching
|   |   `-- yfinance_sp500.py       # S&P 500 benchmark (yfinance fallback)
|   |-- modelling/
|   |   |-- config.py               # Tickers, candidate pairs, date ranges
|   |   |-- cointegration.py        # Engle-Granger two-step test
|   |   |-- spread_analysis.py      # Spread, z-scores, half-life, Hurst
|   |   |-- return_estimation.py    # Historical, EWMA, OU-implied returns
|   |   `-- optimiser.py            # Min-variance, max-Sharpe, frontier
|   |-- backtesting/
|   |   |-- engine.py               # Z-score mean-reversion backtester
|   |   |-- metrics.py              # Sharpe, max drawdown, vol reduction
|   |   `-- benchmarks.py           # Risk-free, B&H, equal-weight, S&P 500
|   |-- dashboard/
|   |   |-- app.py                  # Streamlit application entry point
|   |   `-- components.py           # Plotly visualisation components
|   `-- analysis_notebooks/
|       |-- 01_spread_exploration.ipynb
|       |-- 02_cointegration_validation.ipynb
|       |-- 03_return_estimation.ipynb
|       |-- 04_backtest_results.ipynb
|       `-- figures/                # Exported PDF figures
|-- lseg-data.config.json           # LSEG session config (gitignored)
|-- requirements.txt
`-- README.md
```

D.2 Dependencies

The implementation requires Python 3.10+ with the following key libraries (see Chapter **??** for full details): `pandas`, `numpy`, `statsmodels`, `scipy`, `scikit-learn`, `matplotlib`,`yfinance`. A full list of dependencies is provided in `requirements.txt`.

D.3 Reproducing Results

Complete step-by-step instructions for installation, LSEG data configuration, and running the analysis notebooks and dashboard are provided in Appendix **??** (System Manual). All figures are exported automatically to `src/analysis_notebooks/figures/` as PDF upon notebook execution.

D.4 System Manual

D.4.1 Prerequisites

- Python 3.10 or later (3.11 recommended).
- A valid `lseg-data.config.json` session configuration file in the project root (see Section **??**). Without this file the data client falls back to any locally cached CSVs in `data/raw/`.
- Git (for cloning the repository).

D.4.2 Installation

Clone the repository and install the pinned dependencies into a virtual environment:

```
git clone <repo-url>
cd Asset-Pair-Portfolio-Optimiser
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

All dependencies are pinned in `requirements.txt`; installing into an isolated virtual environment is strongly recommended to avoid version conflicts.

D.4.3 LSEG Data Configuration

The file `lseg-data.config.json` must be placed in the project root before running any notebook that fetches live prices. It is excluded from version control (`.gitignore`) because it contains API session credentials. A template is available from LSEG Workspace under *Settings* → *Developer Tools* → *Python SDK*. The relevant fields are:

```
{
  "sessions": {
    "default": "desktop.workspace",
    "desktop": {
      "workspace": {
        "app-key": "<YOUR_APP_KEY>"
      }
    }
  }
}
```

Once the file is present and the LSEG Workspace desktop application is running, the `refinitiv_client.py` wrapper opens a session automatically on import.

D.4.4 Running the Analysis Notebooks

Execute the notebooks in the following order from the `src/analysis_notebooks/` directory (or from the project root using the paths below):

1. `src/analysis_notebooks/01_spread_exploration.ipynb` — loads prices, screens candidate pairs for cointegration, characterises spreads.
2. `src/analysis_notebooks/02_cointegration_validation.ipynb` — validates cointegration on the out-of-sample period using a rolling window.
3. `src/analysis_notebooks/03_return_estimation.ipynb` — computes historical, EWMA, and OU-implied expected returns and covariance matrices.
4. `src/analysis_notebooks/04_backtest_results.ipynb` — runs the z-score backtester, constructs MPT portfolios, and benchmarks performance.

Each notebook exports all figures to `src/analysis_notebooks/figures/` as PDF files automatically. Notebooks are self-contained; re-running notebook n does not require re-running earlier notebooks provided the intermediate data files in `data/raw/` are present.

D.4.5 Running the Dashboard

The interactive Streamlit dashboard can be launched from the project root:

```
streamlit run src/dashboard/app.py
```

The dashboard opens in the default browser at `http://localhost:8501`. It requires the same `lseg-data.config.json` credentials and will use cached CSVs if the live API is unavailable.

D.4.6 Extending the Codebase

The library packages under `src/` are importable Python modules with no circular dependencies. The dependency graph is strictly layered:

```
modelling/config.py
    <- modelling/cointegration.py
    <- modelling/spread_analysis.py
        <- modelling/return_estimation.py
            <- modelling/optimiser.py
                <- backtesting/engine.py
                <- backtesting/metrics.py
                <- backtesting/benchmarks.py
```

To add a new candidate pair, edit `modelling/config.py` and append the two ticker RICs to the `CANDIDATE_PAIRS` list. To add a new return estimator, implement it in `return_estimation.py` following the same signature as `historical_mean_return` (returns a `pd.Series` of annualised returns indexed by spread label), then pass it as the `expected_returns` argument to `optimiser.optimise_portfolio`.

D.5 Code Listings

The following listings present the core algorithmic components, implemented in Python. Boilerplate and visualisation code are omitted; the full source is available in the project repository.

D.5.1 Cointegration Screening (cointegration.py)

```
def adf_test(series: pd.Series, significance: float = 0.05) -> dict:
    series_clean = series.dropna()
    result = adfuller(series_clean, autolag="AIC")
    return {
        "adf_stat":        result[0],
        "p_value":         result[1],
        "used_lag":        result[2],
        "n_obs":           result[3],
        "critical_values": result[4],
        "is_stationary":   result[1] < significance,
    }


def is_I1(series: pd.Series, significance: float = 0.05) -> dict:
    """
    Check if a series is plausibly I(1):
    non-stationary in levels, stationary in first differences.
    """
    level_result = adf_test(series, significance=significance)
    diff_result  = adf_test(series.diff(), significance=significance)
    return {
        "level":  level_result,
        "diff":   diff_result,
        "is_I1":  (not level_result["is_stationary"])
                   and diff_result["is_stationary"],
    }


def engle_granger_test(
    y: pd.Series, x: pd.Series, significance: float = 0.05
) -> dict:
    """
    Engle-Granger two-step cointegration test.
    Step 1: OLS regression  y = a + b*x + e  (b is the hedge ratio).
    Step 2: ADF test on residuals e via statsmodels coint().
    """
    aligned = pd.concat([y, x], axis=1).dropna().astype(float)
    y_clean, x_clean = aligned.iloc[:, 0], aligned.iloc[:, 1]

    y_I1 = is_I1(y_clean, significance=significance)
    x_I1 = is_I1(x_clean, significance=significance)

    X = sm.add_constant(x_clean)
    model = sm.OLS(y_clean, X).fit()
    intercept, hedge_ratio = model.params

    coint_stat, p_value, critical_values = coint(
        y_clean, x_clean, trend="c", method="aeg", autolag="AIC"
    )

    return {
        "hedge_ratio":    float(hedge_ratio),
        "intercept":      float(intercept),
        "adf_stat":       float(coint_stat),
        "p_value":        float(p_value),
        "critical_values": critical_values,
        "is_cointegrated": bool(
            p_value < significance
            and y_I1["is_I1"] and x_I1["is_I1"]
        ),
    }


def screen_pairs(
    prices_df: pd.DataFrame,
    candidate_pairs: list[tuple[str, str]],
    significance: float = 0.05,
) -> pd.DataFrame:
    """
    Run Engle-Granger on all candidate pairs, testing both orderings
    (y~x and x~y) and keeping the result with the lower p-value.
    """
    rows = []
    for ticker_y, ticker_x in candidate_pairs:
        y, x = prices_df[ticker_y], prices_df[ticker_x]
        result_yx = engle_granger_test(y, x, significance)
        result_xy = engle_granger_test(x, y, significance)

        if result_yx["p_value"] <= result_xy["p_value"]:
            best, dep, indep = result_yx, ticker_y, ticker_x
        else:
            best, dep, indep = result_xy, ticker_x, ticker_y
```

```
        rows.append({
            "y": dep, "x": indep,
            "hedge_ratio":    best["hedge_ratio"],
            "intercept":      best["intercept"],
            "adf_stat":       best["adf_stat"],
            "p_value":        best["p_value"],
            "is_cointegrated": best["is_cointegrated"],
        })

    return pd.DataFrame(rows).sort_values("p_value").reset_index(drop=True)
```

D.5.2 Spread Characterisation (spread_analysis.py)

```
def compute_spread(
    y: pd.Series, x: pd.Series,
    hedge_ratio: float, intercept: float = 0.0,
) -> pd.Series:
    spread = y.astype(float) - hedge_ratio * x.astype(float) - intercept
    spread.name = "spread"
    return spread


def compute_zscore(spread: pd.Series, window: int = 60) -> pd.Series:
    rolling_mean = spread.rolling(window=window).mean()
    rolling_std  = spread.rolling(window=window).std(ddof=1)
    return (spread - rolling_mean) / rolling_std


def compute_half_life(spread: pd.Series) -> float:
    """
    Half-life of mean reversion via OLS on the AR(1) form:
        delta_s = lambda * s_{t-1} + noise
    Half-life = -ln(2) / lambda.
    Returns np.inf if the spread does not appear mean-reverting.
    """
    spread_clean = spread.dropna()
    lag   = spread_clean.shift(1).iloc[1:]
    delta = spread_clean.diff().iloc[1:]

    model = sm.OLS(delta, sm.add_constant(lag)).fit()
    lam = model.params.iloc[1]
    return float(-np.log(2) / lam) if lam < 0 else np.inf


def compute_hurst_exponent(series: pd.Series, max_lag: int = 100) -> float:
    """
    Hurst exponent via the variogram method.
    Fits log(std(increments)) = H * log(lag) + c.
    H < 0.5 -> mean-reverting, H ~ 0.5 -> random walk, H > 0.5 -> trending.
    """
    v = series.dropna().values
    n = len(v)
    max_lag = min(max_lag, n // 2)
    lags = range(2, max_lag + 1)
    tau  = [np.std(v[lag:] - v[:-lag]) for lag in lags]
    poly = np.polyfit(np.log(list(lags)), np.log(tau), 1)
    return float(poly[0])
```

D.5.3 OU-Implied Return Estimator (return_estimation.py)

```
def ou_implied_spread_return(
    spread: pd.Series,
    half_life: float,
    window: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    normalisation: str = "direct",
) -> float:
    """
    OU-implied expected return for a mean-reverting spread.

    Model:  dS = kappa*(theta - S)*dt + sigma*dW
    Expected one-step change:
        E[delta_S] = (theta - S_t) * (1 - exp(-kappa))
    where kappa = ln(2) / half_life.

    With the "direct" normalisation, E[delta_S] is a daily log-return in
    the same units as compute_spread_returns(), so no further scaling is
    needed before passing to the optimiser.
    """
    if half_life <= 0 or np.isinf(half_life):
```

```
        return 0.0

    kappa   = np.log(2) / half_life
    rolling = spread.rolling(window=window)
    theta   = rolling.mean().iloc[-1]
    s_t     = spread.iloc[-1]

    if np.isnan(theta) or np.isnan(s_t):
        return 0.0

    expected_delta = (theta - s_t) * (1 - np.exp(-kappa))

    if normalisation == "std":
        sigma = rolling.std().iloc[-1]
        daily_return = 0.0 if (np.isnan(sigma) or sigma <= 0) \
                       else expected_delta / sigma
    else:  # "direct"
        daily_return = expected_delta

    return float(daily_return * periods_per_year) if annualise \
           else float(daily_return)


def build_ou_implied_returns(
    prices_df: pd.DataFrame,
    coint_pairs: pd.DataFrame,
    window: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    normalisation: str = "direct",
) -> pd.Series:
    """
    Compute OU-implied returns for all cointegrated pairs.
    Log-transforms prices so that E[delta_S] is a log-return directly
    comparable to compute_spread_returns().
    """
    log_prices = np.log(prices_df)
    results = {}
    for _, row in coint_pairs.iterrows():
        label  = f"{row['y']}_vs_{row['x']}"
        spread = compute_spread(
            log_prices[row["y"]], log_prices[row["x"]],
            row["hedge_ratio"], row.get("intercept", 0.0),
        )
        hl = compute_half_life(spread)
        results[label] = ou_implied_spread_return(
            spread, hl, window=window,
            annualise=annualise, periods_per_year=periods_per_year,
            normalisation=normalisation,
        )
    return pd.Series(results, name="ou_implied_return")
```

D.5.4 Markowitz Optimiser (`optimiser.py`)

```
def _portfolio_stats(
    weights: np.ndarray,
    mean_returns: np.ndarray,
    cov_matrix: np.ndarray,
    periods_per_year: int = 252,
) -> tuple[float, float]:
    """Annualised arithmetic return and volatility for a weight vector."""
    log_port_return = np.dot(weights, mean_returns) * periods_per_year
    port_return = np.expm1(log_port_return)   # compound log -> arithmetic
    port_vol    = np.sqrt(
        np.dot(weights, np.dot(cov_matrix, weights)) * periods_per_year
    )
    return port_return, port_vol


def minimum_variance_weights(
    returns: pd.DataFrame,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    l2_reg: float = 0.0,
) -> np.ndarray:
    """Minimum-variance portfolio (long-only, fully invested)."""
    n = returns.shape[1]
    _, cov = _resolve_inputs(returns, cov_matrix=cov_matrix)

    def objective(w):
        return np.dot(w, np.dot(cov, w)) + l2_reg * np.dot(w, w)

    result = minimize(
        objective, np.ones(n) / n,
```

```
        method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
        options={"ftol": 1e-12, "maxiter": 1000},
    )
    return result.x


def maximum_sharpe_weights(
    returns: pd.DataFrame,
    expected_returns: pd.Series | np.ndarray | None = None,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    l2_reg: float = 0.0,
) -> np.ndarray:
    """Maximum Sharpe ratio (tangency) portfolio (long-only)."""
    n = returns.shape[1]
    mean_ret, cov = _resolve_inputs(returns, expected_returns, cov_matrix)

    def neg_sharpe(w):
        port_ret, port_vol = _portfolio_stats(w, mean_ret, cov, periods_per_year)
        if port_vol == 0:
            return 1e6
        return -(port_ret - rf_annual) / port_vol + l2_reg * np.dot(w, w)

    result = minimize(
        neg_sharpe, np.ones(n) / n,
        method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
        options={"ftol": 1e-12, "maxiter": 1000},
    )
    return result.x
```

D.5.5 Pairs Backtesting Engine (engine.py)

```
@dataclass
class BacktestConfig:
    entry_z:              float = 2.0
    exit_z:               float = 0.0
    stop_loss_z:          float = 4.0
    lookback_window:      int   = 60
    transaction_cost_bps: float = 10.0
    initial_capital:      float = 100_000.0

    def __post_init__(self):
        if self.exit_z >= self.entry_z:
            raise ValueError("exit_z must be < entry_z")
        if self.stop_loss_z <= self.entry_z:
            raise ValueError("stop_loss_z must be > entry_z")


class PairsBacktestEngine:

    def _generate_signals(self, zscore: pd.Series) -> pd.Series:
        """
        Position logic:
          +1  long spread  (z < -entry_z, expecting reversion upward)
          -1  short spread (z > +entry_z, expecting reversion downward)
           0  flat
        Stop-loss forces flat whenever |z| >= stop_loss_z.
        """
        cfg = self.config
        positions = pd.Series(0.0, index=zscore.index, name="position")
        pos = 0.0
        for i in range(len(zscore)):
            z = zscore.iloc[i]
            if np.isnan(z):
                positions.iloc[i] = 0.0
                continue
            if abs(z) >= cfg.stop_loss_z and pos != 0.0:
                pos = 0.0
            elif pos == 1.0  and z >= -cfg.exit_z:
                pos = 0.0
            elif pos == -1.0 and z <=  cfg.exit_z:
                pos = 0.0
            elif pos == 0.0  and z <= -cfg.entry_z:
                pos = 1.0
            elif pos == 0.0  and z >=  cfg.entry_z:
                pos = -1.0
            positions.iloc[i] = pos
        return positions
```

```
    def _compute_pnl(
        self,
        y_prices: pd.Series,
        x_prices: pd.Series,
        positions: pd.Series,
        hedge_ratio: float,
    ) -> pd.DataFrame:
        """
        Mark-to-market PnL with proportional transaction costs.
        Full initial capital is deployed into each spread unit:
            n_units = initial_capital / (y_price + |beta| * x_price)
        Daily return:
            r_t = pos_{t-1} * n_units_{t-1} * (dy_t - beta*dx_t) / capital
        """
        cfg = self.config
        dy = y_prices.diff()
        dx = x_prices.diff()
        notional_per_unit = y_prices + abs(hedge_ratio) * x_prices
        n_units  = cfg.initial_capital / notional_per_unit
        spread_pnl = (
            positions.shift(1) * n_units.shift(1) * (dy - hedge_ratio * dx)
        ).fillna(0.0)
        tc = (
            positions.diff().abs().fillna(0.0)
            * cfg.initial_capital
            * (cfg.transaction_cost_bps / 10_000)
        )
        net_pnl = spread_pnl - tc
        return pd.DataFrame({
            "position":         positions,
            "spread_pnl":       spread_pnl,
            "transaction_cost": tc,
            "net_pnl":          net_pnl,
            "strategy_return":  net_pnl / cfg.initial_capital,
        }, index=y_prices.index)
```

D.5.6 Efficient Frontier & Spread-vs-Asset Comparison (`optimiser.py`, `return_estimation.py`)

```
def compute_efficient_frontier(
    returns: pd.DataFrame,
    expected_returns: pd.Series | np.ndarray | None = None,
    cov_matrix: pd.DataFrame | np.ndarray | None = None,
    n_points: int = 50,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    l2_reg: float = 0.0,
) -> pd.DataFrame:
    """
    For a range of target returns (anchored at the min-variance return
    and the single-asset maximum), find the minimum-volatility portfolio
    at each target via SLSQP with a return-equality constraint.
    Returns a DataFrame of (return, volatility, sharpe) for plotting.
    """
    n = returns.shape[1]
    mean_ret, cov = _resolve_inputs(returns, expected_returns, cov_matrix)

    w_min = minimum_variance_weights(returns, cov_matrix, l2_reg=l2_reg)
    ret_min, _ = _portfolio_stats(w_min, mean_ret, cov, periods_per_year)
    ret_max = float(np.max(mean_ret)) * periods_per_year
    if ret_max < ret_min:          # guard: all expected returns negative
        ret_min, ret_max = ret_max, ret_min

    def _min_var_obj(w):
        return np.dot(w, np.dot(cov, w)) + l2_reg * np.dot(w, w)

    records = []
    for target in np.linspace(ret_min, ret_max, n_points):
        constraints = [
            {"type": "eq", "fun": lambda w: np.sum(w) - 1},
            {"type": "eq",
             "fun": lambda w, t=target:
                 np.dot(w, mean_ret) * periods_per_year - t},
        ]
        result = minimize(
            _min_var_obj, np.ones(n) / n, method="SLSQP",
            bounds=[(0.0, 1.0)] * n, constraints=constraints,
            options={"ftol": 1e-12, "maxiter": 1000},
        )
        if result.success:
            port_ret, port_vol = _portfolio_stats(
```

```
                result.x, mean_ret, cov, periods_per_year
            )
            sharpe = (port_ret - rf_annual) / port_vol if port_vol > 0 \
                     else np.nan
            records.append(
                {"return": port_ret, "volatility": port_vol, "sharpe": sharpe}
            )
    return pd.DataFrame(records)


def spread_vs_asset_estimates(
    prices_df: pd.DataFrame,
    coint_pairs: pd.DataFrame,
    method: str = "historical",
    window: int | None = None,
    span: int = 60,
    annualise: bool = True,
    periods_per_year: int = 252,
    cov_estimator: str = "sample",
) -> dict:
    """
    Side-by-side expected-return and covariance estimates for spread-based
    and traditional (asset-level) Markowitz formulations.

    Asset returns are computed as log-returns for consistency with
    compute_spread_returns() (which is a log-return differential).
    Both sides use the same estimator choice (historical, EWMA, or OU;
    sample, Ledoit-Wolf, or OAS).
    """
    spread_rets = build_spread_return_matrix(prices_df, coint_pairs)
    tickers = list(dict.fromkeys(
        coint_pairs["y"].tolist() + coint_pairs["x"].tolist()
    ))
    asset_rets = np.log(prices_df[tickers]).diff().dropna(how="all")

    if method == "ewma":
        spread_mu = ewma_mean_return(spread_rets.dropna(), span=span,
            annualise=annualise, periods_per_year=periods_per_year)
        asset_mu  = ewma_mean_return(asset_rets.dropna(),  span=span,
            annualise=annualise, periods_per_year=periods_per_year)
    elif method == "ou":
        spread_mu = build_ou_implied_returns(prices_df, coint_pairs,
            window=window or 60, annualise=annualise,
            periods_per_year=periods_per_year)
        asset_mu  = historical_mean_return(asset_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)
    else:  # "historical"
        spread_mu = historical_mean_return(spread_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)
        asset_mu  = historical_mean_return(asset_rets.dropna(),
            window=window, annualise=annualise,
            periods_per_year=periods_per_year)

    cov_fn = (
        (lambda r: shrinkage_covariance(r, annualise=annualise,
             periods_per_year=periods_per_year, estimator=cov_estimator))
        if cov_estimator in ("lw", "oas")
        else (lambda r: sample_covariance(r, annualise=annualise,
              periods_per_year=periods_per_year))
    )
    return {
        "spread_returns": spread_rets,
        "spread_mu":      spread_mu,
        "spread_cov":     cov_fn(spread_rets.dropna()),
        "asset_returns":  asset_rets,
        "asset_mu":       asset_mu,
        "asset_cov":      cov_fn(asset_rets.dropna()),
    }
```

D.5.7 Performance Metrics (metrics.py)

```
def compute_ex_post_sharpe_ratio(
    returns: pd.Series,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
) -> float:
    """
    Sharpe (1994) ex-post Sharpe ratio on realised returns.
    Daily excess returns are computed by subtracting the daily risk-free
    rate; the ratio is annualised by sqrt(periods_per_year).
    """
```

```
    if len(returns) < 2:
        raise ValueError("Need at least 2 returns for std dev")
    excess   = returns - (rf_annual / periods_per_year)
    mean_exc = excess.mean()
    std_exc  = excess.std(ddof=1)
    if std_exc == 0:
        return np.nan
    return (mean_exc / std_exc) * np.sqrt(periods_per_year)


def compute_max_drawdown(returns: pd.Series) -> float:
    """
    Maximum drawdown: the largest peak-to-trough decline in the
    compounded equity curve.  Returns a negative fraction.
    """
    if len(returns) < 2:
        raise ValueError("Need at least 2 returns to compute drawdown")
    cum         = (1 + returns).cumprod()
    running_max = cum.cummax()
    drawdowns   = (cum - running_max) / running_max
    return float(drawdowns.min())


def compute_volatility_reduction(
    portfolio_returns: pd.Series,
    benchmark_returns: pd.Series,
    periods_per_year: int = 252,
) -> float:
    """
    Fractional annualised volatility reduction vs a benchmark.
        vol_reduction = 1 - sigma_portfolio / sigma_benchmark
    Positive => portfolio is less volatile; negative => more volatile.
    """
    sigma_p = portfolio_returns.std(ddof=1) * np.sqrt(periods_per_year)
    sigma_b = benchmark_returns.std(ddof=1) * np.sqrt(periods_per_year)
    if sigma_b == 0:
        raise ValueError("Benchmark volatility is zero")
    return float(1 - (sigma_p / sigma_b))
```

D.5.8 Benchmark Suite (benchmarks.py)

```
def compute_benchmark_metrics(
    returns: pd.Series,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
    benchmark_returns: pd.Series | None = None,
) -> dict:
    """
    Compute standard metrics for any return series.  If benchmark_returns
    is provided, also computes volatility_reduction aligned to the common
    trading calendar (inner join on the index).
    """
    ret = returns.dropna()
    out = {
        "sharpe_ratio":          compute_ex_post_sharpe_ratio(
                                     ret, rf_annual, periods_per_year),
        "max_drawdown":          compute_max_drawdown(ret),
        "total_return":          float((1 + ret).prod() - 1),
        "annualised_volatility": float(ret.std(ddof=1)
                                       * np.sqrt(periods_per_year)),
        "volatility_reduction":  np.nan,
    }
    if benchmark_returns is not None:
        common = ret.index.intersection(benchmark_returns.dropna().index)
        if len(common) >= 2:
            try:
                out["volatility_reduction"] = compute_volatility_reduction(
                    ret.reindex(common).dropna(),
                    benchmark_returns.reindex(common).dropna(),
                    periods_per_year,
                )
            except Exception:
                pass
    return out


def historical_mpt_returns(
    train_prices: pd.DataFrame,
    test_prices: pd.DataFrame,
    rf_annual: float = 0.02,
    periods_per_year: int = 252,
) -> pd.Series:
    """
```

```
    Out-of-sample returns of the historical max-Sharpe (traditional MPT)
    portfolio.  Weights are estimated on train_prices using the sample mean
    and covariance of asset returns; fixed weights are then applied to
    test_prices.  Provides the classical Markowitz benchmark for comparison
    with the spread-based strategy.
    """
    train_ret = train_prices.pct_change().dropna()
    mean_ret  = train_ret.mean().values
    cov       = train_ret.cov().values
    n         = len(mean_ret)

    def neg_sharpe(w):
        port_ret = np.dot(w, mean_ret) * periods_per_year
        port_vol = np.sqrt(np.dot(w, np.dot(cov, w)) * periods_per_year)
        return -(port_ret - rf_annual) / port_vol if port_vol > 0 else 1e6

    res = minimize(
        neg_sharpe, np.ones(n) / n, method="SLSQP",
        bounds=[(0.0, 1.0)] * n,
        constraints={"type": "eq", "fun": lambda w: np.sum(w) - 1},
    )
    port_ret = (test_prices[train_prices.columns].pct_change() * res.x
                ).sum(axis=1)
    port_ret.name = "historical_mpt"
    return port_ret.dropna()
```

D.5.9 Trade Logging & Aggregate Metrics (engine.py)

```
def _build_trade_log(
    self,
    positions: pd.Series,
    zscore: pd.Series,
) -> pd.DataFrame:
    """
    Build an event-level trade log from position transitions.
    Each row records the date, direction (entry_long, entry_short, exit),
    the resulting position, and the z-score at that moment.
    """
    changes    = positions.diff().fillna(0.0)
    records    = []
    for date, change in changes[changes != 0.0].items():
        pos_after = positions.loc[date]
        if   pos_after == 0.0:  direction = "exit"
        elif change     > 0:    direction = "entry_long"
        else:                   direction = "entry_short"
        records.append({
            "date":           date,
            "direction":      direction,
            "position_after": pos_after,
            "zscore":         zscore.loc[date]
                              if date in zscore.index else np.nan,
        })
    return pd.DataFrame(records)


def _compute_metrics(
    self,
    daily_returns: pd.Series,
    trades: pd.DataFrame,
    benchmark_returns: pd.Series | None = None,
) -> dict:
    """
    Aggregate performance metrics over the backtest period.

    Win rate is approximated from daily active returns (non-zero position
    days) to avoid double-counting partial holding periods.
    Profit factor = gross profits / gross losses on active days.
    Volatility reduction is measured relative to the equal-weight
    buy-and-hold of the two spread legs.
    """
    valid  = daily_returns.dropna()
    active = valid[valid != 0.0]
    wins   = (active > 0).sum()
    losses = (active < 0).sum()
    win_rate = float(wins / (wins + losses)) if (wins + losses) > 0 \
               else np.nan
    profit_factor = (
        float(active[active > 0].sum() / abs(active[active < 0].sum()))
        if (active < 0).any() else np.inf
    )
    vol_red = np.nan
    if benchmark_returns is not None:
        common = valid.index.intersection(benchmark_returns.dropna().index)
```

```
        if len(common) >= 2:
            try:
                vol_red = compute_volatility_reduction(
                    valid.reindex(common), benchmark_returns.reindex(common)
                )
            except (ValueError, ZeroDivisionError):
                pass
    return {
        "sharpe_ratio":          compute_ex_post_sharpe_ratio(valid),
        "max_drawdown":          compute_max_drawdown(valid),
        "total_return":          float((1 + valid).prod() - 1),
        "num_trades":            len(trades[trades["direction"]
                                            .str.startswith("entry")]),
        "num_round_trips":       len(trades[trades["direction"] == "exit"]),
        "win_rate":              win_rate,
        "profit_factor":         profit_factor,
        "annualised_volatility": float(valid.std(ddof=1) * np.sqrt(252)),
        "volatility_reduction":  vol_red,
    }
```

D.5.10 Data Client with Incremental Cache (refinitiv_client.py)

```
# Cache key: MD5 hash of (sorted tickers, interval, sorted fields).
# A single CSV file grows incrementally as new date ranges are requested.

def _cache_path(tickers: list, interval: str, fields: list) -> Path:
    key    = {"tickers": sorted(tickers), "interval": interval,
              "fields": sorted(fields)}
    digest = hashlib.md5(
        json.dumps(key, sort_keys=True).encode()
    ).hexdigest()[:10]
    fname  = (
        "_".join(sorted(tickers)).replace(".", "-")[:40]
        + f"__{interval}__{digest}.csv"
    )
    return RAW_DATA_DIR / fname


def get_price_timeseries(
    tickers, start=None, end=None, interval="1d",
    fields=None, use_cache=True,
):
    """
    Fetch OHLCV data from LSEG with smart incremental caching.

    On a full cache hit the function returns immediately.  On a partial
    cache hit (the requested window extends beyond the cached range in
    either direction), only the missing segment(s) are fetched, merged,
    and re-persisted.  This minimises LSEG API calls when re-running
    notebooks with slightly different date windows.
    """
    # ... date / ticker / interval normalisation omitted for brevity ...

    path = _cache_path(instruments, interval, fields)

    if use_cache and path.exists():
        cached_df    = _load_cache(path)
        need_before  = start_dt < cached_df.index.min()
        need_after   = end_dt   > cached_df.index.max()

        if not need_before and not need_after:
            return cached_df.loc[start_dt:end_dt]      # full hit

        fetched_parts = []
        if need_before:
            gap_end = pd.Timestamp(np.busday_offset(
                (cached_df.index.min() - timedelta(days=1)).date(),
                0, roll="preceding"
            ))
            if start_dt <= gap_end:
                fetched_parts.append(
                    _fetch_from_lseg(instruments, fields, interval,
                                     start_dt, gap_end)
                )
        if need_after:
            gap_start = cached_df.index.max() + timedelta(days=1)
            if gap_start <= end_dt:
                fetched_parts.append(
                    _fetch_from_lseg(instruments, fields, interval,
                                     gap_start, end_dt)
                )
        if fetched_parts:
            combined = pd.concat([cached_df] + fetched_parts)
```

```
            combined = combined[~combined.index.duplicated(keep="last")]
            combined.sort_index(inplace=True)
            _save_cache(combined, path)
            return combined.loc[start_dt:end_dt]

        return cached_df.loc[start_dt:end_dt]

    df = _fetch_from_lseg(instruments, fields, interval, start_dt, end_dt)
    if use_cache:
        _save_cache(df, path)
    return df


def _clean_frame(df: pd.DataFrame) -> pd.DataFrame:
    """
    Post-fetch cleaning: datetime index, numeric coercion,
    forward-fill up to 5 business days, and drop columns >10% missing.
    """
    df.index = pd.to_datetime(df.index)
    df.index.name = "Date"
    df.dropna(how="all", inplace=True)
    df.sort_index(inplace=True)
    for col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    df = df.ffill(limit=5)
    bad_cols = df.columns[df.isna().mean() > 0.10].tolist()
    if bad_cols:
        warnings.warn(f"Dropping columns with >10% missing: {bad_cols}")
        df.drop(columns=bad_cols, inplace=True)
    return df
```

D.5.11 Universe Configuration (modelling/config.py)

```
# 26 large-cap US equities across 7 sectors, identified by LSEG RIC codes.
TICKERS = [
    "NVDA.O", "AMD.O",  "TSM.N",  "INTC.O",   # Semiconductors
    "KO.N",   "PEP.O",  "COST.O", "TGT.N",    # Consumer Staples
    "JPM.N",  "BAC.N",  "GS.N",   "MS.N",     # Financials
    "XOM.N",  "CVX.N",  "SLB.N",  "HAL.N",    # Energy
    "DAL.N",  "UAL.O",                         # Airlines
    "AMZN.O", "MSFT.O", "META.O", "GOOGL.O",  # Technology / Cloud
    "JNJ.N",  "PFE.N",  "MRK.N",  "ABBV.N",   # Healthcare
]

# 16 sector-homogeneous candidate pairs for cointegration screening.
CANDIDATE_PAIRS = [
    ("NVDA.O", "AMD.O"),  ("NVDA.O", "TSM.N"),
    ("AMD.O",  "INTC.O"), ("TSM.N",  "INTC.O"),
    ("KO.N",   "PEP.O"),  ("COST.O", "TGT.N"),
    ("JPM.N",  "BAC.N"),  ("GS.N",   "MS.N"),
    ("XOM.N",  "CVX.N"),  ("SLB.N",  "HAL.N"),
    ("DAL.N",  "UAL.O"),
    ("AMZN.O", "MSFT.O"), ("META.O", "GOOGL.O"),
    ("AMZN.O", "GOOGL.O"),
    ("JNJ.N",  "PFE.N"),  ("MRK.N",  "ABBV.N"),
]

# In-sample / out-of-sample date split
TRAIN_START = "2018-01-01"
TRAIN_END   = "2023-12-31"   # in-sample (6 years)
TEST_START  = "2024-01-01"   # out-of-sample
TEST_END    = "2025-12-31"

RISK_FREE_RATE = 0.02        # annualised; 3M T-bill average
```